



## Operations Research

Publication details, including instructions for authors and subscription information:  
<http://pubsonline.informs.org>

### Scheduling in Robotic Cells: Heuristics and Cell Design

Hichem Kamoun, Nicholas G. Hall, Chelliah Sriskandarajah,

To cite this article:

Hichem Kamoun, Nicholas G. Hall, Chelliah Sriskandarajah, (1999) Scheduling in Robotic Cells: Heuristics and Cell Design. Operations Research 47(6):821-835. <https://doi.org/10.1287/opre.47.6.821>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact [permissions@informs.org](mailto:permissions@informs.org).

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

© 1999 INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

# SCHEDULING IN ROBOTIC CELLS: HEURISTICS AND CELL DESIGN

HICHEM KAMOUN

*Faculté des Sciences Economiques et de Gestion de Sfax, Tunisia, bh.kamoun@planet.tn*

NICHOLAS G. HALL

*Department of Management Sciences, 2100 New Avenue, The Ohio State University, Columbus, Ohio, 43210-1144, hall.33@osu.edu*

CHELLIAH SRISKANDARAJAH

*School of Management, The University of Texas at Dallas, 2601 N. Floyd Road, Richardson, Texas 75083-0688, Chelliah@utdallas.edu*

(Received January 1995; revisions received April 1996, September 1997; accepted February 1998)

This paper considers scheduling problems arising in robot-served manufacturing cells in which the machines are configured in a flowshop that repetitively produces a family of similar parts. We study the problem of determining the robot move cycle and the part sequence that jointly minimize the average steady-state cycle time required for the repetitive production of a minimal part set, or equivalently maximize the long-run throughput rate. Three earlier related papers provide algorithms, or proofs of intractability, for a variety of cell configurations. We use the intuition developed there to design and test simple heuristic procedures for the part sequencing problem under different robot move cycles in three-machine cells where that problem is intractable. This enables us to develop a heuristic procedure for a general three-machine cell. A methodology for extending this heuristic to four-machine cells is described and tested. Ideas for extension to even larger cells are also discussed. We also describe and test two heuristics for a cell design problem that involves partitioning machines into cells as well as determining the sequence of robot moves and parts. Finally, we provide a list of open research problems.

A cellular manufacturing system consists of a series of cells (each containing flexible machines) with small buffers between the cells (Black 1983, Black and Schroer 1993). Since setup times are reduced to improve flexibility, material handling time and cost become bottlenecks, and efficient material handling becomes very important. Depending on the type of manufacturing facility, estimates ranging from 10% to 80% of total cost have been attributed to material handling (Tompkins et al. 1996). In cellular manufacturing, parts are moved directly from machine to machine, instead of being returned to a storage area between operations. This reduces the number of part moves and the in-process inventory (Askin and Standridge 1993). Hartley (1983) suggests that a cellular manufacturing system is ideally suited to material handling by a robot, in which case it is called a *robotic cell*. Robotic cells with no storage buffers at or between machines, as studied here, are widely used in practice (Asfahl 1985, Miller and Walker 1990).

We consider the natural objective of minimizing the average steady-state cycle time for the repetitive production of minimal part sets (MPSs). For example, if 700 units of A, 400 units of B, and 100 units of C need to be produced, the MPS consists of 12 parts: seven of A, four of B, and one of C. Minimizing cycle time is equivalent to long-run

throughput rate maximization in repetitive steady state manufacture of MPSs. Wittrock (1985) and Matsuo (1990) consider cycle time minimization in a flowshop environment. To achieve this objective, we need to consider the design of the cell, the scheduling of robot moves, and the sequencing of parts to be produced.

Because of its generality and flexibility, we focus on the mobile-robot cell, in which the machines are configured in a flowshop. A three-machine mobile-robot cell is illustrated by Hall et al. (1997,1998). The machines are served by a central robot. Each part being processed passes through the same sequence of locations from an input hopper,  $I$ , through machines  $M_1, \dots, M_m$ , and finally into an output hopper,  $O$ . Since there are no storage buffers, any part in the cell is always either on one of the machines or being handled by the robot. Neither a machine nor the robot can be in possession of more than one part at any time. Processing of any part on any machine is non-preemptive. Robot travel times, and times to pick up, load, unload, and drop parts, are assumed to be independent of the part being moved. This assumption is usually valid in cellular manufacturing systems where a group of similar parts, belonging to the same part family but requiring different processing times, are produced in the same cell.

*Subject classifications:* Production/scheduling: Scheduling robot served manufacturing cells: Sequencing, deterministic, multiple machine: design and computational evaluation of heuristics.

*Area of review:* LOGISTICS AND SUPPLY CHAIN OPERATIONS.

In this paper, we develop and test practical heuristic procedures for difficult scheduling problems in robotic cells. We consider several scheduling problems that have been shown in the literature to be intractable, and provide simple but computationally effective heuristics to solve practical size problems approximately. The intuition which we provide to support these heuristics should help others in the study of even more complex problems. We also study the design of robotic cells within a larger manufacturing system where several machines need to be arranged into cells that are separated by intermediate buffers.

The background and notation required to read this paper are discussed in §1. In §2, we consider three-machine robotic cells and describe and test heuristics for intractable scheduling problems that use specific robot move cycles (as defined in §1). The general three- and four-machine cell problems are studied in §§3 and 4, respectively. Heuristics for larger cells are also discussed in §4. A cell design problem is considered in §5. In §6, we list several open research problems. In §7, we provide a conclusion and some comments about related models.

## 1. BACKGROUND AND NOTATION

Hall (1999) provides an introduction to the use of operations research techniques for the planning, design, control, and analysis of robotic systems. Reviews of the literature related to robotic cells appear in Sethi et al. (1992), Hall et al. (1997), Crama (1997), and Sriskandarajah et al. (1998). Of particular relevance are the works of Drezner and Nof (1984), Wilhelm and Sarin (1985), Nof and Drezner (1986), Wilhelm (1987), and Crama and van de Klundert (1997). Applications of robotic cells, and motivation for using them, are described by Hartley (1983). Logendran and Sriskandarajah (1996) describe various robotic cells and provide an example in which a noncyclic schedule dominates all cyclic schedules. However, since noncyclic schedules usually generate large work-in-process inventory and are hard to control in practice, we follow most of the literature in considering only cyclic schedules.

A robot move cycle is a sequence of activities by the robot that returns to its starting state in steady-state production of a single part-type. Sethi et al. (1992) prove that there are exactly  $m!$  potentially optimal *one-unit robot move cycles* in an  $m$ -machine cell. In such a cycle, the cell returns to the same state after the production of every unit (or part). They further show that any potentially optimal one-unit cycle in an  $m$ -machine robotic cell contains exactly  $m+1$  *basic activities*:

$M_i^-$ : Load a part on  $M_i$ ,  $i = 1, \dots, m$ .

$M_m^+$ : Unload a finished part from  $M_m$ .

Note that the activity  $M_i^+$  (unload a part from  $M_i$ ,  $i = 1, \dots, m$ ) is not required in the basic activities, since it implies loading a part on  $M_{i+1}$ . We now provide brief details on how to interpret a robot move cycle composed of the above basic activities. A convenient point to begin the description

of the cycle is the loading of a part on machine  $M_1$ , denoted by  $M_1^-$ . Let  $\xi$  denote the sequence of basic activities from one occurrence of  $M_1^-$  to the next such occurrence. Since, in a one-unit cycle, every basic activity must be carried out exactly once, there must be exactly one occurrence of each activity  $M_2^-, \dots, M_m^-, M_m^+$  in  $\xi$ . Exactly which machines are occupied by parts at the time when  $M_1^-$  occurs can be determined as follows. For  $i = 2, \dots, m-1$ , machine  $M_i$  is occupied by a part when  $M_1^-$  occurs if and only if  $M_{i+1}^-$  precedes  $M_i^-$  in  $\xi$ . Also, machine  $M_m$  is occupied by a part when  $M_1^-$  occurs if and only if  $M_m^+$  precedes  $M_m^-$  in  $\xi$ . Since exactly which machines are occupied is known, the sequence of basic activities following  $M_1^-$  uniquely determines the ordering of loading and unloading of the machines, and thus also the robot movements between activities. Examples for two- and three-machine robotic cells follow.

Let  $S_{j,m}$  denote the  $j$ th one-unit robot move cycle in an  $m$ -machine cell. Since in an optimal cycle we require that the robot move path is as short as possible, any two consecutive activities uniquely determine the robot moves between those activities. Therefore, a cycle can be uniquely described by a permutation of the above  $m+1$  activities. The following are the available robot move cycles for  $m=2$ :

$$S_{1,2} : \{M_2^+, M_1^-, M_2^-, M_2^+\}, \quad S_{2,2} : \{M_2^+, M_2^-, M_1^-, M_2^+\}.$$

We now interpret these two cycles. In cycle  $S_{1,2}$ , we observe that, following  $M_1^-$  in  $\xi$ , activity  $M_2^-$  precedes  $M_2^+$ . Therefore, we know that machine  $M_2$  is free when  $M_1^-$  occurs. By contrast, in cycle  $S_{2,2}$ , activity  $M_2^+$  precedes  $M_2^-$ . Therefore, we know that machine  $M_2$  is occupied by a part when  $M_1^-$  occurs.

The six robot moves cycles for  $m=3$  can be developed from the above two cycles, by first replacing the starting and ending activities by  $M_3^+$ , as now shown:

$$1. \{M_3^+, M_1^-, M_2^-, M_3^+\}.$$

$$2. \{M_3^+, M_2^-, M_1^-, M_3^+\}.$$

Each of the last two cycles generates three cycles in a three-machine cell, depending upon where the  $M_3^-$  activity is inserted, thus creating a total of six cycles, as now shown:

$$S_{1,3} : \{M_3^+, M_1^-, M_2^-, M_3^-, M_3^+\},$$

$$S_{2,3} : \{M_3^+, M_1^-, M_3^-, M_2^-, M_3^+\},$$

$$S_{3,3} : \{M_3^+, M_3^-, M_1^-, M_2^-, M_3^+\},$$

$$S_{4,3} : \{M_3^+, M_2^-, M_3^-, M_1^-, M_3^+\},$$

$$S_{5,3} : \{M_3^+, M_2^-, M_1^-, M_3^-, M_3^+\},$$

$$S_{6,3} : \{M_3^+, M_3^-, M_2^-, M_1^-, M_3^+\}.$$

We briefly interpret two examples of these cycles. In cycle  $S_{5,3}$ , since  $M_3^-$  precedes  $M_2^-$  in  $\xi$ , machine  $M_2$  must be occupied by a part when  $M_1^-$  occurs; also, since  $M_3^+$  precedes  $M_3^-$ , machine  $M_3$  must be free at that time. In cycle  $S_{6,3}$ , since  $M_3^-$  precedes  $M_2^-$  in  $\xi$ , machine  $M_2$  must be occupied by a part when  $M_1^-$  occurs; also, since  $M_3^+$  precedes  $M_3^-$ , machine  $M_3$  must be occupied by a part at that time.

Each of the above three-machine cycles generates four cycles in a four-machine cell, depending upon where the  $M_4^-$  activity is inserted, thus creating a total of 24 cycles, as shown in Appendix A. The following notation is used to describe a robotic cell.

|                                               |                                                                                                                                                                                                                                         |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $m$                                           | = the number of machines in the cell.                                                                                                                                                                                                   |
| $P_1, \dots, P_k$                             | = the part-types to be produced.                                                                                                                                                                                                        |
| $r_1, \dots, r_k$                             | = the minimal quantities of parts to be produced, where $r_i$ is an integer.                                                                                                                                                            |
| MPS                                           | = a minimal part set consisting of $r_i$ parts of type $P_i$ , $i = 1, \dots, k$ .                                                                                                                                                      |
| $n = r_1 + \dots + r_k$                       | = the total number of parts in an MPS.                                                                                                                                                                                                  |
| $M_1, \dots, M_m$                             | = the machines in the robotic cell, in processing order.                                                                                                                                                                                |
| $I$                                           | = the input hopper.                                                                                                                                                                                                                     |
| $O$                                           | = the output hopper.                                                                                                                                                                                                                    |
| $\sigma$                                      | = the sequence of parts of an MPS that are processed repetitively.                                                                                                                                                                      |
| $P_{\sigma(j)}$                               | = the $j$ th part in the sequence $\sigma$ to be produced.                                                                                                                                                                              |
| $a_{\sigma(j)}$                               | = the processing time of the $j$ th part in sequence $\sigma$ on machine $M_1$ (resp., $M_2$ , (respectively, sequence $\sigma$ on machine $M_1$ (resp., $M_2$ , $M_3$ , $M_4$ )).                                                      |
| $b_{\sigma(j)}, c_{\sigma(j)}, d_{\sigma(j)}$ | = the time taken by additional linear robot movement when traveling between $M_i$ and $M_{i+1}$ , $1 \leq i \leq m-1$ , after rotational movement has been completed.                                                                   |
| $\delta_i$                                    | = the time taken by a rotational robot movement when traveling between $M_{i-1}$ and $M_i$ , $1 \leq i \leq m+1$ , where we let $M_0 = I$ and $M_{m+1} = O$ .                                                                           |
| $\gamma$                                      | = the time saved by not stopping at an intermediate point during a robot movement.                                                                                                                                                      |
| $\varepsilon_1$                               | = the time taken by the robot to pick up a part at $I$ .                                                                                                                                                                                |
| $\varepsilon_{2i}$                            | = the time taken by the robot to load a part onto machine $M_i$ .                                                                                                                                                                       |
| $\varepsilon_{2i+1}$                          | = the time taken by the robot to unload a part from machine $M_i$ .                                                                                                                                                                     |
| $\varepsilon_{2m+2}$                          | = the time taken by the robot to drop a part at $O$ .                                                                                                                                                                                   |
| $(\chi_1, \dots, \chi_m, M_h^j)$              | = the current state of the system, where $\chi_i = \phi$ (respectively, $\Omega$ ) if machine $M_i$ is free (resp., occupied by a part), and the robot has just loaded (respectively, unloaded) machine $M_h$ if $j = -$ (resp., $+$ ). |

We also use the following variables:

$$T_{j(\sigma)} = T_{j\sigma(1)\sigma(2)}^h + T_{j\sigma(2)\sigma(3)}^h + \dots + T_{j\sigma(n)\sigma(1)}^h, \text{ where the average steady-state cycle time required for the repetitive manufacture of an MPS, using part sequence } \sigma \text{ and robot move cycle } S_{j,m}, j \in \{1, \dots, m!\}.$$

|                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $T_{j\sigma(i)\sigma(i+1)}^h$ | = the time between the loading of part $P_{\sigma(i)}$ on machine $M_h$ , and the loading of part $P_{\sigma(i+1)}$ on machine $M_h$ , using robot move cycle $S_{j,m}$ , where $h \in \{1, \dots, m\}$ and $j \in \{1, \dots, m!\}$ , using part sequence $\sigma$ .                                                                                                                                                                                                                                                                                                             |
| $w_h^i$                       | = the waiting time of the robot before unloading part $P_{\sigma(i)}$ at machine $M_h$ , $h \in \{1, \dots, m\}$ . Note that $w_h^i$ depends on the robot move cycle being used as well as the part sequence.                                                                                                                                                                                                                                                                                                                                                                     |
| $\mathcal{N}$                 | = the total number of Type 2 robot waiting times in a robot move cycle. Note that the robot performs two kinds of waiting in a cell. We refer to these as Type 1 and Type 2 waiting. In Type 1 waiting, the robot loads a part on a machine and waits there throughout processing, so the waiting time is exactly the processing time of that part on that machine. In Type 2 waiting, the robot loads a part and performs other activities during its processing. The robot returns to unload the part and waits for $w_h^i$ (which may be zero) time until processing finishes. |
| $E_0$                         | = a state of the robot move cycle in which all machines except $M_i$ are free, for some $1 < i \leq m$ , and the robot has just completed loading a part on $M_i$ , and is about to move to $I$ .                                                                                                                                                                                                                                                                                                                                                                                 |
| $E^*$                         | = a state of the robot cycle in which the robot has just completed the loading of a part on a machine, and all other machines are free.                                                                                                                                                                                                                                                                                                                                                                                                                                           |

The robot travel time between locations  $x$  and  $y$  is denoted by  $\ell(x, y)$ . For example, in a three-machine mobile-robot cell,  $\ell(I, M_1) = \delta_1$ ,  $\ell(M_1, M_2) = \delta_0 + \delta_2$ ,  $\ell(M_2, M_3) = \delta_0 + \delta_3$ ,  $\ell(M_3, O) = \delta_4$ ,  $\ell(O, I) = 2\delta_0 + \delta_1 + \delta_2 + \delta_3 + \delta_4 - 3\gamma$ ,  $\ell(O, M_1) = 2\delta_0 + \delta_2 + \delta_3 + \delta_4 - 2\gamma$ , and  $\ell(M_2, I) = \delta_0 + \delta_1 + \delta_2 - \gamma$ . The velocity of the robot arm as it moves from one point to another has three different phases: acceleration, constant velocity, and deceleration. For instance, the travel time from  $I$  to  $M_2$  directly is  $\gamma$  units less than the travel time from  $I$  to  $M_1$  plus that from  $M_1$  to  $M_2$ , since the latter requires additional acceleration and deceleration phases. Similarly, the travel time to move directly from  $I$  to  $O$  is  $3\gamma$  units less than the travel time from  $I$  to  $M_1$  plus those from  $M_1$  to  $M_2$ ,  $M_2$  to  $M_3$ , and  $M_3$  to  $O$ . We assume that processing times ( $a_i, b_i, c_i, d_i, \dots$ ) and robot activity times ( $\delta_i, \gamma, \varepsilon_i$ ) are deterministic and constant over time.

The standard classification scheme for scheduling problems (Graham et al. 1979) uses  $\psi_1|\psi_2|\psi_3$ , where  $\psi_1$  indicates the scheduling environment,  $\psi_2$  describes the job characteristics or restrictions, and  $\psi_3$  defines the objective function to be minimized. Hall et al. (1997) extend this scheme to classify scheduling problems arising in robotic cells, as follows. Under  $\psi_1$ , we have

$MRCm$  = a mobile-robot cell with  $m$  machines.

Under  $\psi_2$ , we might have

$k$  = the number of part-types produced.

$q\text{-unit}(s)$  = the problem is being solved over robot move cycles that produce  $q$  units.

$S_j$  = robot move cycle  $S_j$  alone is being used.

Under  $\psi_3$ , we have

$C_t$  = the average steady state cycle time for the repetitive manufacture of an MPS.

Thus, for example,  $MRC3|k \geq 2, S_2|C_t$  denotes the minimization of cycle time for the repetitive production of an MPS in a three-machine mobile-robot cell producing multiple part-types, using robot move cycle  $S_{2,3}$ . Since  $m$  specifies the number of machines in the cell, the second subscript on  $S$  is omitted wherever there is no ambiguity.

## 2. THREE-MACHINE CELL, MULTIPLE PART-TYPE PROBLEMS

In multiple part-type production, we need to produce a set of parts (MPS) repetitively in the same part sequence,  $\sigma$ . We consider a *production cycle*, which produces a set of parts repetitively using  $v$  repetitions of a robot move cycle, where  $v$  is the number of parts in the set. Hall et al. (1998) show that a single MPS (i.e.,  $v = n$ ) is sufficient to define a production cycle in a steady state for all robot move cycles except for  $S_{2,3}$ . They also show that, for  $n$  even, the  $S_{2,3}$  cycle requires a single MPS to define a production cycle in a steady state. However, for  $n$  odd, two MPS cycles are needed to define a production cycle.

In this section, we consider two part-sequencing problems under the condition that the parts are to be produced by a specific one-unit robot move cycle,  $S_{2,3}$  or  $S_{6,3}$ . The recognition versions of these problems are shown by Hall et al. (1998) to be unary  $NP$ -complete (see Garey and Johnson 1979 for definitions). It should be noted that the part-sequencing problems under the other four one-unit robot move cycles are all solvable in polynomial time (Hall et al. 1997), thus the need for heuristics does not arise there. We show below how to derive the cycle time formulas for cycles  $S_{2,3}$  and  $S_{6,3}$ .

### 2.1. A Heuristic for Problem $MRC3|k \geq 2, S_2|C_t$

As mentioned above, robot move cycle  $S_{2,3}$  might require two MPSs to define a production cycle in a steady state. Consequently, we set  $v$ , the number of parts in a production cycle, equal to  $2n$ . Thus, a given part sequence  $\sigma$  means a sequence of  $2n$  parts in which the first  $n$  parts corresponding to an MPS have the same order as the last  $n$  parts. Consequently, the production of MPSs occurs repetitively in the same sequence, as required. We now derive the cycle time for producing a set of parts (MPS) under cycle  $S_{2,3}$  in  $MRC3|k \geq 2, 1\text{-unit}|C_t$ .

Starting from the initial state  $E = (\phi, \Omega, \Omega, M_2^-)$ , where the robot has just loaded part  $P_{\sigma(i)}$  on machine  $M_2$ ,  $M_3$  is occupied by part  $P_{\sigma(i-1)}$ , and  $M_1$  is free, robot move sequence  $S_{2,3}$  includes the following activities, with times shown in parentheses: move from  $M_2$  to  $M_3$ :  $(\delta_0 + \delta_3)$ ,

if necessary wait for part  $P_{\sigma(i-1)}$  to be processed at  $M_3$ :  $(w_3^{i-1})$ , unload  $P_{\sigma(i-1)}$  from  $M_3$ :  $(\varepsilon_7)$ , move to  $O$ :  $(\delta_4)$ , drop  $P_{\sigma(i-1)}$  at  $O$ :  $(\varepsilon_8)$ , move to  $I$ :  $(2\delta_0 + \delta_1 + \delta_2 + \delta_3 + \delta_4 - 3\gamma)$ , pick up part  $P_{\sigma(i+1)}$  at  $I$ :  $(\varepsilon_1)$ , move to  $M_1$ :  $(\delta_1)$ , load  $P_{\sigma(i+1)}$  on  $M_1$ :  $(\varepsilon_2)$ , move to  $M_2$ :  $(\delta_0 + \delta_2)$ , if necessary wait for part  $P_{\sigma(i)}$  to be processed at  $M_2$ :  $(w_2^i)$ , unload  $P_{\sigma(i)}$  from  $M_2$ :  $(\varepsilon_5)$ , move to  $M_3$ :  $(\delta_0 + \delta_3)$ , load  $P_{\sigma(i)}$  on  $M_3$ :  $(\varepsilon_6)$ , move to  $M_1$ :  $(2\delta_0 + \delta_2 + \delta_3 - \gamma)$ , if necessary wait for part  $P_{\sigma(i+1)}$  to be processed at machine  $M_1$ :  $(w_1^{i+1})$ , unload  $P_{\sigma(i+1)}$  from  $M_1$ :  $(\varepsilon_3)$ , move to  $M_2$ :  $(\delta_0 + \delta_2)$ , and load  $P_{\sigma(i+1)}$  on  $M_2$ :  $(\varepsilon_4)$ . Thus, we derive the following cycle time expression for  $S_{2,3}$ :

$$T_{2\sigma(i)\sigma(i+1)}^2 = \sum_{i=1}^8 \varepsilon_i + 8\delta_0 + 2\delta_1 + 4\delta_2 + 4\delta_3 + 2\delta_4 - 4\gamma + w_1^{i+1} + w_2^i + w_3^{i-1},$$

where  $w_1^{i+1} = \max\{0, a_{2\sigma(i+1)} - \beta_2 - w_2^i\}$ ,  $w_2^i = \max\{0, b_{2\sigma(i)} - \beta_2 - w_3^{i-1}\}$ , and  $w_3^{i-1} = \max\{0, c_{2\sigma(i-1)} - \beta_2 - w_1^{i-1}\}$ . Substituting for  $w_1^{i+1}$  and  $w_2^i$ , we obtain  $T_{2(\sigma)} = \frac{1}{2} \sum_{i=1}^v T_{2\sigma(i)\sigma(i+1)}^2 = \frac{1}{2} v \alpha_2 + \frac{1}{2} \sum_{i=1}^v \max\{\beta_2, a_{2\sigma(i+1)}, b_{2\sigma(i)} - w_3^{i-1}\} + \frac{1}{2} \sum_{i=1}^v w_3^{i-1}$ , where  $w_3^{i-1} = \max\{0, c_{2\sigma(i-1)} - \max\{\beta_2, a_{2\sigma(i)} - \max\{0, b_{2\sigma(i-1)} - \beta_2 - w_3^{i-2}\}\}\}$ , where we define  $b_{2\sigma(v+1)} = b_{2\sigma(1)}$ ,  $a_{2\sigma(v+1)} = a_{2\sigma(1)}$ ,  $a_{2\sigma(v+2)} = a_{2\sigma(2)}$ , and where  $\alpha_2 = 4\delta_0 + 2\delta_2 + 2\delta_3 - \gamma$ ,  $\beta_2 = \sum_{i=1}^8 \varepsilon_i + 4\delta_0 + 2\delta_1 + 2\delta_2 + 2\delta_3 + 2\delta_4 - 3\gamma$ ,  $a_{2\sigma(i)} = a_{\sigma(i)} + \sum_{i=1}^4 \varepsilon_i + \sum_{i=7}^8 \varepsilon_i + 2\delta_1 + 2\delta_4 - 2\gamma$ ,  $b_{2\sigma(i)} = b_{\sigma(i)} + \sum_{i=3}^6 \varepsilon_i$ , and  $c_{2\sigma(i)} = c_{\sigma(i)} + \sum_{i=1}^2 \varepsilon_i + \sum_{i=5}^8 \varepsilon_i + 2\delta_1 + 2\delta_4 - 2\gamma$ . Note that  $(\alpha_2 + \beta_2)$  is the total robot activity (loading, unloading, and moving) time in the robot move cycle. The robot activity time does not include robot waiting time at machines. Also,  $\beta_2$  overestimates the total robot activity time between the time of loading a part on a machine until the time at which the robot returns to the same machine to unload the part. Since  $\beta_2$  is required in the robot waiting time formulas, new processing times  $a_{2\sigma(i)}$ ,  $b_{2\sigma(i)}$ , and  $c_{2\sigma(i)}$  are defined as above to make the formulas as concise as possible.

We now propose a heuristic for this problem. For convenience, we use  $T_{2\sigma(i)\sigma(i+1)}^3 = \alpha_2 + \beta_2 + w_1^{i+1} + w_2^{i+1} + w_3^i$ , where  $w_1^{i+1} = \max\{0, a_{2\sigma(i+1)} - \beta_2 - w_2^i\}$ ,  $w_2^{i+1} = \max\{0, b_{2\sigma(i+1)} - \beta_2 - w_3^i\}$ , and  $w_3^i = \max\{0, c_{2\sigma(i)} - \beta_2 - w_1^{i+1}\}$ . These formulas can be derived as above. The intuition underlying the heuristic is developed from the above cycle time formulas. We convert the three-machine problem into a series of two-machine problems and exploit the solvability of two-machine robotic cells. To do so, we ignore a single machine  $M_h$ ,  $h \in \{1, 2, 3\}$ , by setting the processing times of all parts on  $M_h$  equal to zero. In this way, as can be seen below, we obtain three efficiently solvable two-machine problems.

We set the processing times of all parts on  $M_1$  equal to zero, i.e.,  $a_i = 0$  for all  $i$ . Thus, we have  $w_1^i = 0$  for all  $i$ . Using the above formula, we obtain  $T_{2\sigma(i)\sigma(i+1)}^3 = \alpha_2 + \max\{\beta_2, b_{2\sigma(i+1)}, c_{2\sigma(i)}\}$ , which can be interpreted as the distance from city  $i$  to  $i+1$  in a traveling salesman problem (TSP), which is equivalent to a two-machine no-wait

flowshop (Hall and Sriskandarajah 1996). A TSP with this cost structure can be solved efficiently by the algorithm of Gilmore and Gomory (1964), as in Step 1 of the following heuristic. Similarly, by setting  $b_i = 0$  (respectively,  $c_i = 0$ ) for all  $i$ , we obtain  $T_{2\sigma(i)\sigma(i+1)}^3 = \alpha_2 + \max\{\beta_2, a_{2\sigma(i+1)}, c_{2\sigma(i)}\}$  (resp.,  $T_{2\sigma(i)\sigma(i+1)}^2 = \alpha_2 + \max\{\beta_2, a_{2\sigma(i+1)}, b_{2\sigma(i)}\}$ ), which is again solved by the algorithm of Gilmore and Gomory (1964) in Step 2 (resp., Step 3). Finally, we compare the three solutions and select one with smallest cycle time. The idea of our heuristic is similar to that proposed by Campbell et al. (1970), who convert an  $m$  machine flowshop problem into  $(m - 1)$  problems with two machines.

Now we describe our heuristic for cycle  $S_{2,3}$ . Note that since the sequence of the first  $n$  parts is the same as the sequence of the last  $n$  parts, the heuristic considers only the sequence of the first  $n$  parts.

### Heuristic $S_2$ cycle

*Step 0:* Given  $a_i, b_i, c_i$ ,  $1 \leq i \leq n$ ,  $\delta_0, \dots, \delta_4, \varepsilon_1, \dots, \varepsilon_8$ , and  $\gamma$ .  $\beta_2 = \sum_{i=1}^8 \varepsilon_i + 2\sum_{i=0}^4 \delta_i + 2\delta_0 - 3\gamma$ .

*Step 1:*  $e_i = b_i + \varepsilon_3 + \varepsilon_4 + \varepsilon_5 + \varepsilon_6$ ,  $i = 1, \dots, n$ ,  
 $f_i = \max\{\beta_2, c_i + \varepsilon_1 + \varepsilon_2 + \varepsilon_5 + \varepsilon_6 + \varepsilon_7 + \varepsilon_8 + 2\delta_1 + 2\delta_4 - 2\gamma\}$ ,  $i = 1, \dots, n$ .

Apply the algorithm of Gilmore and Gomory (1964) to the instance  $\{e_i, f_i\}$ ,  $i = 1, \dots, n$ , obtain the part sequence  $\sigma_1$ , and calculate  $T_{2(\sigma_1)}$ .  $C_h = T_{2(\sigma_1)}$ ,  $\sigma^* = \sigma_1$ .

*Step 2:*  $e_i = a_i + \varepsilon_1 + \varepsilon_2 + \varepsilon_3 + \varepsilon_4 + \varepsilon_7 + \varepsilon_8 + 2\delta_1 + 2\delta_4 - 2\gamma$ ,  $i = 1, \dots, n$ ,

$f_i = \max\{\beta_2, c_i + \varepsilon_1 + \varepsilon_2 + \varepsilon_5 + \varepsilon_6 + \varepsilon_7 + \varepsilon_8 + 2\delta_1 + 2\delta_4 - 2\gamma\}$ ,  $i = 1, \dots, n$ .

Apply the algorithm of Gilmore and Gomory (1964) to the instance  $\{e_i, f_i\}$ ,  $i = 1, \dots, n$ , obtain the part sequence  $\sigma_2$ , and calculate  $T_{2(\sigma_2)}$ .

If  $T_{2(\sigma_2)} < C_h$ , then  $C_h = T_{2(\sigma_2)}$  and  $\sigma^* = \sigma_2$ .

*Step 3:*  $e_i = a_i + \varepsilon_1 + \varepsilon_2 + \varepsilon_3 + \varepsilon_4 + \varepsilon_7 + \varepsilon_8 + 2\delta_1 + 2\delta_4 - 2\gamma$ ,  $i = 1, \dots, n$ ,

$f_i = \max\{\beta_2, b_i + \varepsilon_3 + \varepsilon_4 + \varepsilon_5 + \varepsilon_6\}$ ,  $i = 1, \dots, n$ .

Apply the algorithm of Gilmore and Gomory (1964) to the instance  $\{e_i, f_i\}$ ,  $i = 1, \dots, n$ , obtain the part sequence  $\sigma_3$ , and calculate  $T_{2(\sigma_3)}$ .

If  $T_{2(\sigma_3)} < C_h$ , then  $C_h = T_{2(\sigma_3)}$  and  $\sigma^* = \sigma_3$ . Terminate.

In the above heuristic, the cycle time  $T_{2(\sigma)}$  for given  $\sigma$  is found by an efficient algorithm, **FindTime2**, developed by Hall et al. (1998).

The following mixed integer program is used to test the performance of Heuristic  $S_2$ cycle. From the above cycle time derivation, we have

$$T_{2(\sigma)} = \frac{1}{2}v(\alpha_2 + \beta_2) + \frac{1}{2} \sum_{q=1}^3 \sum_{i=1}^v w_q^i,$$

where  $\alpha_2 = 4\delta_0 + 2\delta_2 + 2\delta_3 - \gamma$ ,  $w_1^{i+1} = \max\{0, a_{2\sigma(i+1)} - w_2^i - \beta_2\}$ ,  $w_2^i = \max\{0, b_{2\sigma(i)} - w_3^{i-1} - \beta_2\}$ , and  $w_3^{i-1} =$

$\max\{0, c_{2\sigma(i-1)} - w_1^i - \beta_2\}$ . Note that  $v(\alpha_2 + \beta_2)$  is a constant.

Based on these equations, we can write the formulation associated with  $MRC3|k \geq 2, S_2|C_t$  as the following mixed integer programming problem (MIP2):

$$\min \sum_{q=1}^3 \sum_{i=1}^v w_q^i,$$

subject to

$$\sum_{j=1}^n X_{ij} = 1, \quad i = 1, \dots, n,$$

$$\sum_{i=1}^n X_{ij} = 1, \quad j = 1, \dots, n,$$

$$X_{ij} = X_{i+n, j+n}, \quad i = 1, \dots, n, j = 1, \dots, n,$$

$$w_1^i \geq \sum_{j=1}^n a_{2j} X_{ij} - \beta_2 - w_2^{i-1}, \quad i = 2, \dots, v+1,$$

$$w_2^i \geq \sum_{j=1}^n b_{2j} X_{ij} - \beta_2 - w_3^{i-1}, \quad i = 2, \dots, v+1,$$

$$w_3^i \geq \sum_{j=1}^n c_{2j} X_{ij} - \beta_2 - w_1^{i+1}, \quad i = 1, \dots, v,$$

$$w_q^i \geq 0, \quad i = 1, \dots, v, q = 1, 2, 3,$$

$$X_{ij} \in \{0, 1\}, \quad i = 1, \dots, v, j = 1, \dots, v.$$

Here  $X_{ij} = 1$  if part  $j$  is scheduled in the  $i$ th position of the sequence  $\sigma$ ,  $X_{ij} = 0$  otherwise, for  $i = 1, \dots, v$ ,  $j = 1, \dots, v$ . Also, in the above formulation, we let  $w_1^1 = w_1^{v+1}$  and  $w_2^1 = w_2^{v+1}$ , and when  $v = 2n$  we let  $a_{2j} = a_{2, j+n}$ ,  $b_{2j} = b_{2, j+n}$ , and  $c_{2j} = c_{2, j+n}$ ,  $j = 1, \dots, n$ . The third set of constraints is not needed when  $v = n$ . Our computational results are discussed in §2.3.

### 2.2. A Heuristic for Problem $MRC3|k \geq 2, S_6|C_t$

As mentioned above, a single MPS is sufficient to define a production cycle in a steady state in cycle  $S_{6,3}$  (Hall et al. 1998). We now derive the cycle time for producing a set of parts (MPS) under cycle  $S_{6,3}$  in  $MRC3|k \geq 2, 1 - unit|C_t$ .

Starting from the initial state  $E = (\Omega, \Omega, \Omega, M_1^-)$ , where the robot has just loaded part  $P_{\sigma(i)}$  on machine  $M_1$ , and  $M_2$  and  $M_3$  are occupied by parts  $P_{\sigma(i-1)}$  and  $P_{\sigma(i-2)}$ , respectively, robot move sequence  $S_{6,3}$  includes the following activities, with times shown in parentheses: move from  $M_1$  to  $M_3$ :  $(2\delta_0 + \delta_2 + \delta_3 - \gamma)$ , if necessary wait for part  $P_{\sigma(i-2)}$  to be processed at  $M_3$ :  $(w_3^{i-2})$ , unload  $P_{\sigma(i-2)}$  from  $M_3$ :  $(\varepsilon_7)$ , move to  $O$ :  $(\delta_4)$ , drop  $P_{\sigma(i-2)}$  at  $O$ :  $(\varepsilon_8)$ , move to  $M_2$ :  $(\delta_0 + \delta_3 + \delta_4 - \gamma)$ , if necessary wait for part  $P_{\sigma(i-1)}$  to be processed at  $M_2$ :  $(w_2^{i-1})$ , unload  $P_{\sigma(i-1)}$  from  $M_2$ :  $(\varepsilon_5)$ , move to  $M_3$ :  $(\delta_0 + \delta_3)$ , load  $P_{\sigma(i-1)}$  on  $M_3$ :  $(\varepsilon_6)$ , move to  $M_1$ :  $(2\delta_0 + \delta_2 + \delta_3 - \gamma)$ , if necessary wait for part  $P_{\sigma(i)}$  to be processed at  $M_1$ :  $(w_1^i)$ , unload  $P_{\sigma(i)}$  from  $M_1$ :  $(\varepsilon_3)$ , move to  $M_2$ :  $(\delta_0 + \delta_2)$ , load  $P_{\sigma(i)}$  on  $M_2$ :  $(\varepsilon_4)$ , move to

$I$ :  $(\delta_0 + \delta_1 + \delta_2 - \gamma)$ , pick up part  $P_{\sigma(i+1)}$  at  $I$ :  $(\varepsilon_1)$ , move to  $M_1$ :  $(\delta_1)$ , and load  $P_{\sigma(i+1)}$  on  $M_1$ :  $(\varepsilon_2)$ . Thus, we derive the following expression for  $S_{6,3}$ :

$$T_{6\sigma(i)\sigma(i+1)}^1 = \sum_{i=1}^8 \varepsilon_i + 8\delta_0 + 2\delta_1 + 4\delta_2 + 4\delta_3 + 2\delta_4 - 4\gamma \\ + w_1^i + w_2^{i-1} + w_3^{i-2},$$

where  $w_1^i = \max\{0, a_{6\sigma(i)} - \beta_6 - w_2^{i-1} - w_3^{i-2}\}$ ,  $w_2^{i-1} = \max\{0, b_{6\sigma(i-1)} - \beta_6 - w_3^{i-2}\}$ , and  $w_3^{i-2} = \max\{0, c_{6\sigma(i-2)} - \beta_6 - w_1^{i-1}\}$ . Substituting for  $w_1^i$ ,  $w_2^{i-1}$ , and  $w_3^{i-2}$ , we obtain  $T_{6\sigma} = \sum_{i=1}^n T_{6\sigma(i)\sigma(i+1)}^1 = n\alpha_6 + \sum_{i=1}^n \max\{\beta_6, c_{6\sigma(i-2)} - w_1^{i-1}, b_{6\sigma(i-1)}, a_{6\sigma(i)}\}$ , and  $w_1^{i-1} = \max\{0, a_{6\sigma(i-1)} - \max\{\beta_6, b_{6\sigma(i-2)}, c_{6\sigma(i-3)} - w_1^{i-2}\}\}$ , where  $\alpha_6 = 2\delta_0$ ,  $\beta_6 = 6\delta_0 + 2\delta_1 + 4\delta_2 + 4\delta_3 + 2\delta_4 + \sum_{i=1}^8 \varepsilon_i - 4\gamma$ ,  $a_{6\sigma(i)} = a_{\sigma(i)} + 2\delta_1 + 2\delta_2 + \sum_{i=1}^4 \varepsilon_i - \gamma$ ,  $b_{6\sigma(i)} = b_{\sigma(i)} + 2\delta_0 + 2\delta_2 + 2\delta_3 + \sum_{i=3}^6 \varepsilon_i - \gamma$ , and  $c_{6\sigma(i)} = c_{\sigma(i)} + 2\delta_3 + 2\delta_4 + \sum_{i=5}^8 \varepsilon_i - \gamma$ .

We now propose a heuristic for this problem. The intuition underlying the heuristic is similar to that for Heuristic  $S_2$ cycle in §2.1. We set the processing times of all parts on  $M_1$  equal to zero, i.e.,  $a_i = 0$  for all  $i$ . Thus, we have  $w_1^i = 0$  for all  $i$ . Using the above formula, we obtain  $T_{6\sigma(i)\sigma(i+1)}^1 = \alpha_6 + \max\{\beta_6, b_{6\sigma(i-1)}, c_{6\sigma(i-2)}\}$ , which defines a TSP that can be solved efficiently by the algorithm of Gilmore and Gomory (1964), as in Step 1 of the following heuristic. Similarly, by setting  $c_i = 0$  for all  $i$ , we obtain  $T_{6\sigma(i)\sigma(i+1)}^1 = \alpha_6 + \max\{\beta_6, a_{6\sigma(i)}, b_{6\sigma(i-1)}\}$ , which is again solved by the algorithm of Gilmore and Gomory (1964) in Step 2. However, setting  $b_i = 0$  for all  $i$  does not give rise to a TSP formulation. Finally, we compare the two solutions and select one with smaller cycle time.

### Heuristic $S_6$ cycle

**Step 0:** Given  $a_i, b_i, c_i$ ,  $1 \leq i \leq n$ ,  $\delta_0, \dots, \delta_4, \varepsilon_1, \dots, \varepsilon_8$ , and  $\gamma$ .  $\alpha_6 = 2\delta_0$ .  
 $\beta_6 = 6\delta_0 + 2\delta_1 + 4\delta_2 + 4\delta_3 + 2\delta_4 + \sum_{i=1}^8 \varepsilon_i - 4\gamma$ .  
**Step 1:**  $e_i = b_i + \varepsilon_3 + \varepsilon_4 + \varepsilon_5 + \varepsilon_6 + 2\delta_0 + 2\delta_2 + 2\delta_3 - \gamma$ ,  $i = 1, \dots, n$ ,  
 $f_i = \max\{\beta_6, c_i + \varepsilon_5 + \varepsilon_6 + \varepsilon_7 + \varepsilon_8 + 2\delta_3 + 2\delta_4 - \gamma\}$ ,  $i = 1, \dots, n$ .  
 Apply the algorithm of Gilmore and Gomory (1964) to the instance  $\{e_i, f_i\}$ ,  $i = 1, \dots, n$ , obtain the part sequence  $\sigma_1$ , and calculate  $T_{6(\sigma_1)}$ .  
 $C_h = T_{6(\sigma_1)}$ ,  $\sigma^* = \sigma_1$ .  
**Step 2:**  $e_i = a_i + \varepsilon_1 + \varepsilon_2 + \varepsilon_3 + \varepsilon_4 + 2\delta_1 + 2\delta_2 - \gamma$ ,  $i = 1, \dots, n$ ,  
 $f_i = \max\{\beta_6, b_i + \varepsilon_3 + \varepsilon_4 + \varepsilon_5 + \varepsilon_6 + 2\delta_0 + 2\delta_2 + 2\delta_3 - \gamma\}$ ,  $i = 1, \dots, n$ .  
 Apply the algorithm of Gilmore and Gomory (1964) to the instance  $\{e_i, f_i\}$ ,  $i = 1, \dots, n$ , obtain the part sequence  $\sigma_2$ , and calculate  $T_{6(\sigma_2)}$ .  
 If  $T_{6(\sigma_2)} < C_h$ , then  $C_h = T_{6(\sigma_2)}$ ,  $\sigma^* = \sigma_2$ . Terminate.

In the above heuristic, the cycle time  $T_{6(\sigma)}$  for given  $\sigma$  is found by an efficient algorithm, **FindTime6**, developed by Hall et al. (1998).

**Table 1.**  $S_2$ cycle performance vs. MIP2.

| $n$ | $\chi$ | Opt. | MRPE  | MRPEO | WRPE   |
|-----|--------|------|-------|-------|--------|
| 5   | 50     | 18   | 2.528 | 3.950 | 10.884 |
| 7   | 50     | 2    | 4.792 | 4.991 | 13.808 |
| 10  | 50     | 0    | 5.539 | 5.539 | 10.729 |

**Table 2.**  $S_6$ cycle performance vs. MIP6.

| $n$ | $\chi$ | Opt. | MRPE  | MRPEO | WRPE   |
|-----|--------|------|-------|-------|--------|
| 5   | 50     | 30   | 1.686 | 4.215 | 12.266 |
| 7   | 50     | 9    | 3.854 | 4.700 | 11.796 |
| 10  | 50     | 4    | 4.405 | 4.788 | 11.334 |

We use a mixed integer program, MIP6, which is similar to MIP2, to test the performance of Heuristic  $S_6$ cycle. Our computational results are discussed in §2.3.

### 2.3. Computational Testing of Heuristics

Heuristics  $S_2$ cycle and  $S_6$ cycle were coded in the Turbo Pascal language, and MIP2 and MIP6 were solved optimally using the software package CPLEX 3.0. We consider the following data as fixed:  $\varepsilon_i = 1.0$ ,  $i = 1, \dots, 8$ ,  $\delta_0 = 0.0$ ,  $\delta_i = 4.0$ ,  $i = 1, \dots, 4$ , and  $\gamma = 0.0$ . Processing times were uniformly generated integers in the range 1, ..., 100. All our data were generated randomly using  $k = n$ , which allows, but does not require, identical parts within an MPS.

Let  $C^q$  (respectively,  $C_h^q$ ) denote the cycle time delivered by MIP2 or MIP6 (resp.,  $S_2$ cycle or  $S_6$ cycle) for instance  $q$ . Tables 1 and 2 compare the performance of  $S_2$ cycle and  $S_6$ cycle with the optimal solutions obtained by solving MIP2 and MIP6, respectively. The third column, Opt., shows the number of optimal solutions found by  $S_2$ cycle or  $S_6$ cycle over  $\chi$  randomly generated problem instances in each row, where  $\chi = 50$ . The fourth and sixth columns show the mean relative percentage error (MRPE) and worst (i.e., maximum) relative percentage error (WRPE), respectively, of  $S_2$ cycle or  $S_6$ cycle over 50 problem instances. These are defined as  $\text{MRPE} = 100 \sum_{q=1}^{\chi} (C_h^q - C^q) / \chi C^q$  and  $\text{WRPE} = 100 \max_{1 \leq q \leq \chi} \{(C_h^q - C^q) / C^q\}$ . The fifth column, MRPEO, is similar to MRPE but calculated only over instances where an optimal solution was not found. The CPU time required by the heuristics is very small on a Pentium 60/90 MHz computer model. As the size (and therefore complexity) of the instance increases, the number of optimal solutions found by the heuristics decreases, and the mean relative percentage error slowly increases.

### 3. HEURISTICS FOR PROBLEM $MRC3|k \geq 2|C_i$

This problem is a completely general three-machine robotic cell in which all possible robot move cycles are

considered. Hall et al. (1998) show that its recognition version is unary *NP*-complete. For two machine cells, Hall et al. (1997) provide an efficient algorithm, **MinCycle**, that simultaneously optimizes the robot move and part sequencing problems. Aneja and Kamoun (1999) describe a more efficient algorithm, Improved MinCycle, which is used below to obtain lower bounds for  $MRC3|k \geq 2|C_t$  and in §5 to design cells.

Here we describe a computationally effective heuristic for problem  $MRC3|k \geq 2|C_t$ . Recall that the six one-unit robot move cycles that are available in this cell,  $S_{1,3}, \dots, S_{6,3}$ , define a part sequencing problem that we can solve, either optimally by the polynomial time algorithms proposed by Hall et al. (1997), or approximately using Heuristics  $S_2$ **cycle** and  $S_6$ **cycle**. We begin by describing heuristics for the part sequencing problem in two new cycles. Both are composed of parts of three one-unit cycles, each of which can be formulated as a TSP.

**THEOREM 1.** *Problem  $MRC3|k \geq 2, S_1, S_3, S_4|C_t$  can be formulated as a TSP.*

**PROOF.** Note that  $E = (\phi, \phi, \Omega, M_3^-)$  is a state common to cycles  $S_{1,3}, S_{3,3}$ , and  $S_{4,3}$ . Starting from  $E$ , the robot has three ways in which it can produce a single part and return to the same state. These consist of following one of the three robot move cycles  $S_{1,3}, S_{3,3}$ , or  $S_{4,3}$ . Starting from the common state  $E = (\phi, \phi, \Omega, M_3^-)$ , the following cycle time expressions can be derived similarly to the one for cycle  $S_{6,3}$  in §2.2. If  $S_{1,3}$  is used, we have

$$T_{1\sigma(i)\sigma(i+1)}^3 = \alpha_1 + c_{\sigma(i)} + a_{\sigma(i+1)} + b_{\sigma(i+1)},$$

where  $\alpha_1 = 4\delta_0 + 2 \sum_{i=1}^4 \delta_i + \sum_{i=1}^8 \varepsilon_i - 3\gamma$ .

Alternatively, if  $S_{3,3}$  is used, we have

$$T_{3\sigma(i)\sigma(i+1)}^3 = \alpha_3 + \max\{\beta_3 + a_{\sigma(i+1)}, \\ + b_{3\sigma(i+1)} + a_{\sigma(i+1)}, c_{3\sigma(i)}\},$$

where  $\alpha_3 = 2\delta_0 + 2\delta_3 + \varepsilon_5 + \varepsilon_6 - \gamma$ ,  $\beta_3 = 4\delta_0 + 2 \sum_{i=1}^4 \delta_i + \sum_{i=1}^4 \varepsilon_i + \varepsilon_7 + \varepsilon_8 - 2\gamma$ ,  $b_{3\sigma(i)} = b_{\sigma(i)} + 2\delta_0 + 2\delta_1 + 2\delta_2 + \sum_{i=1}^4 \varepsilon_i - \gamma$ , and  $c_{3\sigma(i)} = c_{\sigma(i)} + 2\delta_4 + \varepsilon_7 + \varepsilon_8$ .

Finally, if  $S_{4,3}$  is used, we have

$$T_{4\sigma(i)\sigma(i+1)}^3 = \alpha_4 + b_{\sigma(i+1)} + \max\{\beta_4, c_{4\sigma(i)}, a_{4\sigma(i+1)}\},$$

where  $\alpha_4 = 4\delta_0 + 2\delta_2 + 2\delta_3 + \varepsilon_3 + \varepsilon_4 + \varepsilon_5 + \varepsilon_6 - 2\gamma$ ,  $\beta_4 = 4\delta_0 + 2\delta_1 + 2\delta_2 + 2\delta_3 + 2\delta_4 + \varepsilon_1 + \varepsilon_2 + \varepsilon_7 + \varepsilon_8 - 3\gamma$ ,  $a_{4\sigma(i)} = a_{\sigma(i)} + 2\delta_1 + \varepsilon_1 + \varepsilon_2$ , and  $c_{4\sigma(i)} = c_{\sigma(i)} + 2\delta_4 + \varepsilon_7 + \varepsilon_8$ .

Thus, the cycle time expression becomes

$$T_\sigma = \sum_{i=1}^n \Gamma_{\sigma(i)\sigma(i+1)},$$

where  $\Gamma_{\sigma(i)\sigma(i+1)} = \min\{T_{1\sigma(i)\sigma(i+1)}^3, T_{3\sigma(i)\sigma(i+1)}^3, T_{4\sigma(i)\sigma(i+1)}^3\}$  can be interpreted as the distance from city  $i$  to city  $i+1$  in a TSP formulation.  $\square$

We next consider problem  $MRC3|k \geq 2, S_1, S_4, S_5|C_t$ .

**THEOREM 2.** *Problem  $MRC3|k \geq 2, S_1, S_4, S_5|C_t$  can be formulated as a TSP, where  $\Gamma_{\sigma(i)\sigma(i+1)} = \min\{T_{1\sigma(i)\sigma(i+1)}^2, T_{4\sigma(i)\sigma(i+1)}^2, T_{5\sigma(i)\sigma(i+1)}^2\}$ ,*

$$T_{1\sigma(i)\sigma(i+1)}^2 = \alpha_1 + b_{\sigma(i)} + c_{\sigma(i)} + a_{\sigma(i+1)},$$

$$T_{4\sigma(i)\sigma(i+1)}^2 = \alpha_4 + b_{\sigma(i)} + \max\{\beta_4, c_{4\sigma(i)}, a_{4\sigma(i+1)}\},$$

$$T_{5\sigma(i)\sigma(i+1)}^2 = \alpha_5 + \max\{\beta_5 + c_{\sigma(i)}, b_{5\sigma(i)} + c_{\sigma(i)}, a_{5\sigma(i+1)}\},$$

and where  $\alpha_5 = 2\delta_0 + 2\delta_2 + \varepsilon_3 + \varepsilon_4 - \gamma$ ,  $\beta_5 = 4\delta_0 + 2\delta_1 + 2\delta_2 + 2\delta_3 + 2\delta_4 + \varepsilon_1 + \varepsilon_2 + \varepsilon_5 + \varepsilon_6 + \varepsilon_7 + \varepsilon_8 - 2\gamma$ ,  $a_{5\sigma(i)} = a_{\sigma(i)} + 2\delta_1 + \varepsilon_1 + \varepsilon_2$ , and  $b_{5\sigma(i)} = b_{\sigma(i)} + 2\delta_0 + 2\delta_3 + 2\delta_4 + \varepsilon_5 + \varepsilon_6 + \varepsilon_7 + \varepsilon_8 - \gamma$ .

**PROOF.** Similar to that of Theorem 1.

We now summarize our heuristic for  $MRC3|k \geq 2|C_t$ . It uses the solution procedures described earlier for all six one-unit cycles, as well as heuristic solutions using GENIUS (Gendreau et al. 1992) to the TSPs formulated in Theorems 1 and 2. The smallest cycle time is chosen. The heuristic thus solves the problem over all the robot move cycles that generate part-sequencing problems known to be solvable in polynomial time, either optimally or very effectively by a heuristic method.

### Heuristic ThreeCell

*Step 1:* Use the algorithms from Hall et al. (1997) to solve  $MRC3|k \geq 2, S_1|C_t$  (respectively,  $MRC3|k \geq 2, S_3|C_t$ ,  $MRC3|k \geq 2, S_4|C_t$ ,  $MRC3|k \geq 2, S_5|C_t$ ). Let  $\sigma_1$  (resp.,  $\sigma_3, \sigma_4, \sigma_5$ ) denote the associated part sequences, and  $C_1$  (resp.,  $C_3, C_4, C_5$ ) represent their cycle times.

*Step 2:* Use Heuristic  $S_2$ **cycle** for  $MRC3|k \geq 2, S_2|C_t$ , and Heuristic  $S_6$ **cycle** for  $MRC3|k \geq 2, S_6|C_t$ . Let  $\sigma_2$  (respectively,  $\sigma_6$ ) denote the associated part sequences, and  $C_2$  (resp.,  $C_6$ ) represent their cycle times.

*Step 3:* Use Heuristic GENIUS to solve  $MRC3|k \geq 2, S_1, S_3, S_4|C_t$  and  $MRC3|k \geq 2, S_1, S_4, S_5|C_t$ . Let  $\sigma'$  (respectively,  $\sigma''$ ) denote the associated part sequences, and  $C''$  (resp.,  $C''$ ) represent their cycle times.

*Step 4:* Find  $C_h = \min\{C_1, C_2, C_3, C_4, C_5, C_6, C', C''\}$ , and its associated robot move cycle and part sequence. Terminate.

**ThreeCell** was coded in the Turbo Pascal language and run on a Pentium 60/90 MHz computer. We generated random problem instances using four data sets, which are now described. In all our data generation we let  $k = n$ , which allows, but does not require, identical parts within an MPS.

**data1:**  $\varepsilon_i = 1.0$ ,  $i = 1, \dots, 8$ ,  $\delta_0 = 0.0$ ,  $\delta_i = 4.0$ ,  $i = 1, \dots, 4$ ,  $\gamma = 0.0$ . The processing times are generated independently from the uniform distribution over the integers 20, ..., 99.

**data2:** as in data1, except that the processing times are generated independently from the uniform distribution over the integers 10, ..., 49.

**data3:** as in data1, except that the processing times of all the parts on the last machine are generated independently from the uniform distribution over the integers 1, ..., 9, and the processing times on the other machines are generated

**Table 3.** ThreeCell performance vs. LBCT3 using data1.

| $n$ | $pmin$ | $pmax$ | $\chi$ | MRPE   | WRPE   | $O_6$ | $O'$ | $O''$ | CPU (sec) |
|-----|--------|--------|--------|--------|--------|-------|------|-------|-----------|
| 6   | 20     | 99     | 50     | 4.258  | 11.264 | 50    | 0    | 0     | 0.119     |
| 10  | 20     | 99     | 50     | 8.647  | 14.167 | 50    | 0    | 0     | 0.340     |
| 20  | 20     | 99     | 50     | 10.616 | 16.726 | 50    | 0    | 0     | 1.298     |
| 50  | 20     | 99     | 50     | 12.651 | 17.040 | 50    | 0    | 0     | 6.917     |

**Table 4.** ThreeCell performance vs. LBCT3 using data3.

| $n$ | $pmin$ | $pmax$ | $\chi$ | MRPE   | WRPE   | $O_6$ | $O_5$ | $O'$ | $O''$ | CPU (sec) |
|-----|--------|--------|--------|--------|--------|-------|-------|------|-------|-----------|
| 6   | 10     | 49     | 50     | 9.927  | 16.286 | 48    | 2     | 0    | 2     | 0.081     |
| 10  | 10     | 49     | 50     | 10.829 | 14.648 | 49    | 1     | 0    | 1     | 0.302     |
| 20  | 10     | 49     | 50     | 11.975 | 15.722 | 50    | 0     | 0    | 0     | 1.239     |
| 50  | 10     | 49     | 50     | 12.967 | 14.908 | 50    | 0     | 0    | 0     | 6.982     |

**Table 5.** ThreeCell performance vs. LBCT3 using data4.

| $n$ | $pmin$ | $pmax$ | $\chi$ | MRPE  | WRPE  | $O_6$ | $O'$ | $O''$ | CPU (sec) |
|-----|--------|--------|--------|-------|-------|-------|------|-------|-----------|
| 6   | 1      | 9      | 50     | 2.568 | 5.998 | 0     | 25   | 25    | 0.084     |
| 10  | 1      | 9      | 50     | 2.763 | 5.158 | 0     | 24   | 26    | 0.349     |
| 20  | 1      | 9      | 50     | 2.752 | 4.137 | 0     | 26   | 24    | 1.372     |
| 50  | 1      | 9      | 50     | 3.152 | 3.998 | 0     | 28   | 22    | 9.449     |

**Table 6.** ThreeCell performance vs. LBCT3 using data5.

| $n$ | $pmin$ | $pmax$ | $\chi$ | MRPE  | WRPE  | $O_1$ | $O'$ | $O''$ | CPU (sec) |
|-----|--------|--------|--------|-------|-------|-------|------|-------|-----------|
| 6   | 1      | 4      | 50     | 0.000 | 0.000 | 50    | 50   | 50    | 0.083     |
| 10  | 1      | 4      | 50     | 0.000 | 0.000 | 50    | 50   | 50    | 0.346     |
| 20  | 1      | 4      | 50     | 0.000 | 0.000 | 50    | 50   | 50    | 1.318     |
| 50  | 1      | 4      | 50     | 0.000 | 0.000 | 50    | 50   | 50    | 8.424     |

independently from the uniform distribution over the integers  $10, \dots, 49$ .

**data4:** as in data1, except that the processing times are generated from  $1, \dots, 9$ .

**data5:** as in data1, except that the processing times are generated from  $1, \dots, 4$ .

Because of the difficulty of finding optimal solutions to this problem, we tested the performance of **ThreeCell** against a lower bound on the optimal cycle time computed as follows.

**Procedure LBCT3**( $a_i, b_i, c_i, i = 1, \dots, n, \delta_0, \delta_1, \delta_2, \delta_3, \delta_4, \gamma$ )

*Step 1:* Call MinCycle( $a_i, b_i, i = 1, \dots, n, \delta_0, \delta_1, \delta_2, \delta_3, \gamma$ ), giving cycle time  $C^*$

$LB_1 = C^*$

Call MinCycle( $b_i, c_i, i = 1, \dots, n, \delta_0, \delta_2, \delta_3, \delta_4, \gamma$ ), giving cycle time  $C^*$

$LB_2 = C^*$

$LB_3 = n\alpha_1 + \sum_{i=1}^n \min\{a_i + b_i + c_i, \min\{a_i + b_i + \delta_0 + \delta_3, a_i + c_i + \delta_0 + \delta_{2,3}, b_i + c_i + \delta_0 + \delta_2\}, \min\{a_i + 2\delta_0 + \delta_{2,3} + \delta_3, b_i + 2\delta_0 + \delta_2 + \delta_3, c_i + 2\delta_0 + \delta_2 + \delta_{2,3}\}, 3\delta_0 + \delta_2 + \delta_{2,3} + \delta_3\}$ ,

where  $\delta_{i,j} = \min\{\delta_i, \delta_j\}$  and  $\alpha_1 = 4\delta_0 + 2\sum_{i=1}^4 \delta_i + \sum_{i=1}^8 \varepsilon - 3\gamma$ .

*Step 2:*  $LB = \max\{LB_1, LB_2, LB_3\}$ . Terminate.

In **LBCT3**,  $LB_1$  is the cycle time delivered by **MinCycle**( $a_i, b_i, i = 1, \dots, n, \delta_0, \delta_1, \delta_2, \delta_3, \gamma$ ).  $LB_2$ , similarly, is the cycle time delivered by **MinCycle**( $b_i, c_i, i = 1, \dots, n, \delta_0, \delta_2, \delta_3, \delta_4, \gamma$ ).  $LB_3$  is a lower bound on the robot activity times in the cell using any robot move cycles to process the  $n$  parts ( $n\alpha_1$ ), plus some additional robot activity time and robot waiting time, as now explained. If the robot waits for part  $P_i$  to be processed at each of the three machines, the contribution of these robot moves to the cycle time is simply  $\alpha_1 + a_i + b_i + c_i$ . If, in processing part  $P_i$ , the robot waits for the part at exactly one machine (for example,  $M_1$ ), then an additional robot move requiring time (for example,  $2\delta_0 + \delta_{2,3} + \delta_3$ ) is added to a lower bound on the robot activity times in the cell plus a lower bound on the waiting time (for example,  $a_i$ ) of the robot at the machine. Similarly, if in processing part  $P_i$ , the robot waits for the part at exactly two machines, i.e., the robot leaves part  $P_i$  on a machine and performs another task while the part is being processed at only one machine (for example,  $M_3$ ), then an additional robot move requiring time (for example,  $\delta_0 + \delta_3$ ) is added to a lower bound on the robot activity times in the cell plus a lower bound on the waiting times (for example,  $a_i + b_i$ ) of the robot at the two machines. Finally, if the robot does not wait at any of the machines in processing part  $P_i$ , the bound on the additional robot activity time is simply  $(3\delta_0 + \delta_2 + \delta_{2,3} + \delta_3)$ .

The results of testing **ThreeCell** against **LBCT3** on 200 random problem instances generated using data1 (respectively, data3, data4, data5) appear in Table 3 (resp., Table 4, Table 5, Table 6). Much of the information there follows the format of Tables 1 and 2. We use  $pmin$  and  $pmax$  to denote the lower and upper ends of the range of processing times, respectively. In addition, in column 7 of Table 3 (respectively, 8, 9),  $O_6$  (resp.,  $O'$ ,  $O''$ ), represents the number of instances for which the best cycle time found is given by  $C_6$  (resp.,  $C'$ ,  $C''$ ). Similar information is provided in Tables 4, 5, and 6. The mean CPU time on a Pentium 60/90 MHz computer model over the  $\chi$  problem instances in each row is also shown, where  $\chi = 50$ . We note that when robot activity times are small compared to processing times, as is the case in data1 and data2, the machines become the bottlenecks. Hence, it is efficient for the robot to load the parts on each machine and move to perform other activities, as in cycle  $S_6$  which dominates for the problem instances in data1 and data2. Because the results for data2 are similar to those for data1, we do not provide a table for data2.

The results obtained for data3 look similar to the results for data1; however, the combined cycles also compete here as the processing times on the last machine are small. Thus, the cycles (i.e., the combined cycles and  $S_5$ ) that load the part on the last machine and stay there during processing are potential candidates for best solution (see Table 4). However, when robot activity times are large compared to processing times, as in data4 and data5, the two combined cycles usually provide better solutions. This is because the combined cycles enable the robot to switch from one cycle to another by loading a part and either waiting at the machine or moving to perform other activities in the cell.

In Table 6 using data5, the two combined cycles have identical performance for all the 200 problems tested. Cycle  $S_1$  is common to both combined cycles. In data5, processing times are very small compared to robot move times, thus  $S_1$  is the most efficient cycle, since it loads the part and waits with it at each machine. Although we tested two combined cycles, the solutions given by both of them used only cycle  $S_1$  for processing all the parts in the MPS. This resulted in equal performance for both cycles. Heuristic solutions typically attained our lower bound here, since the robot loads and waits at each machine for all parts, as in the first component of  $LB_3$  in Procedure **LBCT3**, which tends to dominate the other lower bound components when processing times are small compared to robot activity times.

#### 4. HEURISTICS FOR LARGE CELLS

There are exactly  $m!$  potentially optimal one-unit robot move cycles,  $S_{j,m}$  for  $j \in \{1, \dots, m!\}$ , in an  $m$ -machine cell. Sriskandarajah et al. (1998) classify these cycles according to the complexity of their associated part sequencing problems, as follows:

**Class U:** sequence independent (i.e., trivially solvable).

**Class V:** problems which can be formulated as a TSP.

**Class W:** Unary  $NP$ -hard, but not having TSP structure.

They characterize these cycles as follows. A  $U$ -cycle is the unique cycle where the robot loads the part and waits at all the machines during its processing. The  $V$ -cycles contain an  $E_0$  state in which all machines expect  $M_i$  are free, for some  $1 < i \leq m$ , and the robot has just completed loading a part on  $M_i$ , and is about to move to the input hopper,  $I$ . The remaining cycles are classified as  $W$ -cycles. The authors further divide Class V into two subclasses:

**Class V1:** TSP special cases (solvable in polynomial time using the algorithm of Gilmore and Gomory, 1964).

**Class V2:** unary  $NP$ -hard TSP models.

In  $V1$ -cycles an  $E_0$  state occurs, and at exactly two machines, the robot loads a part and leaves it to perform other operations, where at least one of those machines must be either the first or the last machine in the cell.

The results obtained by Sriskandarajah et al. (1998) can be summarized as follows. In an  $m$ -machine robotic cell with  $m \geq 2$ , there are  $m!$  robot move cycles, of which

- (i) one  $U$ -cycle defines a trivially solvable part sequencing problem.
- (ii)  $Z$   $V$ -cycles define a part sequencing problem which can be formulated as a TSP, where

$$Z = \sum_{t=1}^{\lfloor \frac{m}{2} \rfloor} \binom{m}{2t}.$$

- (iii)  $(m! - 1 - Z)$   $W$ -cycles define a part sequencing problem which in general cannot be formulated directly as a TSP, and which is unary  $NP$ -hard.

They further show that  $(2m - 3)$   $V1$ -cycles define a part-sequencing problem that is solvable in  $O(n \log n)$  time. The  $(Z - 2m + 3)$   $V2$ -cycles are unary  $NP$ -hard. For example, in a four-machine cell,  $S_{1,4}$  is a  $U$ -cycle. Cycles  $S_{4,4}, S_{11,4}, S_{13,4}, S_{14,4}$ , and  $S_{17,4}$  are  $V1$ -cycles. Cycles  $S_{9,4}$  and  $S_{10,4}$  are  $V2$ -cycles. The remaining 16 cycles are all  $W$ -cycles. See Appendix A for details.

We now illustrate how one can extend the ideas in Heuristic **ThreeCell** to a four-machine cell. A similar extension is also possible for cells with more than four machines. Before we describe our overall heuristic for a four-machine cell, we describe a heuristic for  $W$ -cycles in such a cell. Let  $\mathcal{N}$  denote the total number of Type 2 robot waiting times in a robot move cycle.

Given a  $W$ -cycle  $S_{j,m}$  for  $m > 3$ , we propose the following heuristic. All the equivalence results between  $W$ -cycles mentioned in the heuristic are proved by Sriskandarajah et al. (1998). For this purpose, they classify the  $W$ -cycles as follows: cycles having  $\mathcal{N} < m$  and others having  $\mathcal{N} = m$ . The case  $\mathcal{N} = m$  is further classified into two types as follows. In Type A, cycle  $S_{j,m}$ :  $\{M_m^+, \dots, M_p^-, M_i^-, M_1^-, M_{i+1}^-, M_r^-, \dots, M_m^+\}$  contains the sequence of machine loadings  $M_i^-, M_1^-, M_{i+1}^-$  for some  $2 \leq i \leq m$ , where we let  $M_{m+1}^- = M_m^+$ , whereas Type B does not contain the sequence of machine loadings  $M_i^-, M_1^-, M_{i+1}^-$ .

### Heuristic *W*cycle

*Step 1:* Given  $m, j$  and  $S_{j,m}$  (a  $W$ -cycle). Set  $\xi = S_{j,m}$ .

*Step 2:* Find  $\mathcal{N}$  for cycle  $\xi$ .

*Step 3:* If  $\mathcal{N} < m$ , then there exists a machine  $M_i$  such that  $\xi = \{M_m^+, \dots, M_p^-, M_i^-, M_{i+1}^-, M_r^-, \dots, M_m^+\}$  contains the sequence of machine loadings  $M_i^-, M_{i+1}^-$ . Set the processing times of all the parts on  $M_i$  equal to zero. The resulting problem is equivalent to the problem under cycle  $\xi' = \{M_m^+, \dots, M_p^-, M_{i+1}^-, M_r^-, \dots, M_m^+\}$ , which is a  $W$ -cycle is an  $(m-1)$  machine cell that does not contain  $M_i$ . Set  $m = m-1$  and  $\xi = \xi'$ . If  $m = 3$ , then go to Step 8. Otherwise, go to Step 2.

*Step 4:* If  $\mathcal{N} = m$ , then identify whether cycle  $\xi$  is Type A or Type B.

*Step 5:* If  $\xi$  is Type A, then go to Step 6. Otherwise,  $\xi = \{M_m^+, \dots, M_p^-, M_1^-, M_r^-, \dots, M_m^+\}$  contains the sequence of machine loadings  $M_p^-, M_1^-, M_r^-$ , where  $r \neq p+1$ . Set the processing times of all the parts on  $M_1$  equal to zero. The resulting problem is equivalent to the problem under cycle  $\xi' = \{M_m^+, \dots, M_p^-, M_r^-, \dots, M_m^+\}$ , which is a  $W$ -cycle in an  $(m-1)$  machine cell that does not contain  $M_1$ . Set  $m = m-1$  and  $\xi = \xi'$ . If  $m = 3$ , then go to Step 8. Otherwise, go to Step 2.

*Step 6:* If  $m > 4$ , then go to Step 7. Otherwise, identify a machine  $M_i$  such that  $\xi = \{M_m^+, \dots, M_p^-, M_i^-, M_1^-, M_{i+1}^-, M_r^-, \dots, M_m^+\}$  contains the sequence of machine loadings  $M_i^-, M_1^-, M_{i+1}^-$ . Set the processing times of all the parts on  $M_i$  equal to zero. The resulting problem is equivalent to the problem under cycle  $\xi' = \{M_m^+, \dots, M_p^-, M_{i+1}^-, M_1^-, M_r^-, \dots, M_m^+\}$ , which is a  $W$ -cycle in an  $(m-1)$  machine cell that does not contain  $M_i$ . Set  $m = m-1$ ,  $\xi = \xi'$  and go to Step 8. Otherwise, go to Step 2.

*Step 7:* Identify a machine  $M_i$  such that  $\xi = \{M_m^+, \dots, M_p^-, M_i^-, M_1^-, M_{i+1}^-, M_r^-, \dots, M_m^+\}$  contains the sequence of machine loadings  $M_i^-, M_1^-, M_{i+1}^-$ . Set the processing times of all the parts on  $M_1$  and  $M_i$  equal to zero. The resulting problem is equivalent to the problem under cycle  $\xi' = \{M_m^+, \dots, M_p^-, M_{i+1}^-, M_r^-, \dots, M_m^+\}$ , which is a  $W$ -cycle in an  $(m-2)$  machine cell that does not contain  $M_i$  and  $M_1$ . Set  $m = m-2$  and  $\xi = \xi'$ . If  $m = 3$ , then go to Step 8. Otherwise, go to Step 2.

*Step 8:* Solve the part sequencing problem under cycle  $\xi$ . Note that  $\xi$  is either  $S_{2,3}$  or  $S_{6,3}$ , thus part sequence  $\sigma_j$  is obtained by solving the problem as shown in §2.1 or 2.2, respectively. Let  $C_j$  denote the cycle time for  $\sigma_j$  in the original  $m$ -machine problem. Terminate.

In Step 8, the cycle time  $C_j$  for  $\sigma_j$  in the original  $m$ -machine problem is found using a CPLEX 3.0 package, since it can be formulated as a linear program (Srisankandarajah et al. 1998).

We now identify the multiple-unit cycles in a four-machine cell in which the part-sequencing problem can be formulated as a TSP. These cycles are combinations of the one-unit  $U$ -cycle and the  $V$ -cycles for which the part-sequencing problem can be formulated as a TSP.

**THEOREM 3.** *Problem  $MRC4|k \geq 2, S_1, S_4, S_{11}, S_{14}|C_t$  can be formulated as a TSP, where  $\Gamma_{\sigma(i)\sigma(i+1)} = \min\{T_{1\sigma(i)\sigma(i+1)}^4, T_{4\sigma(i)\sigma(i+1)}^4, T_{11\sigma(i)\sigma(i+1)}^4, T_{14\sigma(i)\sigma(i+1)}^4\}$ .*

**PROOF.** Similar to that of Theorem 1.

**THEOREM 4.** *Problem  $MRC4|k \geq 2, S_1, S_{13}, S_{14}, S_{17}|C_t$  can be formulated as a TSP, where  $\Gamma_{\sigma(i)\sigma(i+1)} = \min\{T_{1\sigma(i)\sigma(i+1)}^2, T_{13\sigma(i)\sigma(i+1)}^2, T_{14\sigma(i)\sigma(i+1)}^2, T_{17\sigma(i)\sigma(i+1)}^2\}$ .*

**PROOF.** Similar to that of Theorem 1.

**THEOREM 5.** *Problem  $MRC4|k \geq 2, S_1, S_9, S_{10}, S_{11}, S_{13}, S_{14}|C_t$  can be formulated as a TSP, where  $\Gamma_{\sigma(i)\sigma(i+1)} = \min\{T_{1\sigma(i)\sigma(i+1)}^3, T_{9\sigma(i)\sigma(i+1)}^3, T_{10\sigma(i)\sigma(i+1)}^3, T_{11\sigma(i)\sigma(i+1)}^3, T_{13\sigma(i)\sigma(i+1)}^3, T_{14\sigma(i)\sigma(i+1)}^3\}$ .*

**PROOF.** Similar to that of Theorem 1.

**REMARK 1.** The part sequencing problem associated with any  $U$ - or  $V$ -cycle can be formulated as a TSP. All  $U$ - and  $V$ -cycles have an  $E^*$  state, in which the robot has just completed the loading of a part on a machine, and all other machines are free. In a  $U$ -cycle, an  $E^*$  state occurs at all machines. But, in any  $V$ -cycle, it occurs at one or more machines from  $M_2, \dots, M_m$ . In order to formulate a combined cycle as a TSP, the component cycles must have an  $E^*$  state on the same machine. We are interested in identifying *maximal* feasible combined cycles, i.e., those that allow as many components as possible, since they offer the greatest number of scheduling options, and thus dominate other combined cycles. It is easy to see that there exists at least one  $V$ -cycle with an  $E^*$  state at any machine  $M_i$ ,  $2 \leq i \leq m$ . The  $U$ -cycle can be combined with these  $V$ -cycles, to create a single maximal combined cycle with  $E^*$  at any machine  $M_i$ ,  $2 \leq i \leq m$ . Since there are  $m-1$  such machines, there are  $m-1$  maximal combined cycles with a TSP formulation in an  $m$ -machine robotic cell.

We now summarize our heuristic for  $MRC4|k \geq 2|C_t$ . It uses the solution procedures described earlier for all 24 one-unit cycles, as well as heuristic solutions to the TSPs in Theorem 3, 4 and 5. The smallest cycle time is chosen. We thus solve the problem over all the cycles with part-sequencing problems which are polynomially solvable or have an effective heuristic.

### Heuristic FourCell

*Step 1:* Let  $C_1$  denote the cycle time of any part sequence for the  $U$ -cycle,  $S_1$ .

*Step 2:* Use the algorithm of Gilmore and Gomory (1964) to solve the part-sequencing problem for all  $V1$ -cycles. Let  $C_j$  denote the cycle time for  $V1$ -cycle  $S_j$ ,  $j \in \{4, 11, 13, 14, 17\}$ .

*Step 3:* Use Heuristic GENIUS of Gendreau et al. (1992) to solve the part-sequencing problem for all  $V2$ -cycles. Let  $C_j$  denote the cycle time for  $V2$ -cycle  $S_j$ ,  $j \in \{9, 10\}$ .

**Table 7.** FourCell performance vs. LBCT4 using data1.

| $n$ | $pmin$ | $pmax$ | $\chi$ | MRPE   | WRPE   | $O_{24}$ | $O'$ | $O''$ | $O'''$ | CPU (sec) |
|-----|--------|--------|--------|--------|--------|----------|------|-------|--------|-----------|
| 6   | 20     | 99     | 50     | 8.044  | 18.085 | 50       | 0    | 0     | 0      | 5.532     |
| 10  | 20     | 99     | 50     | 12.089 | 19.434 | 50       | 0    | 0     | 0      | 5.995     |
| 20  | 20     | 99     | 50     | 15.184 | 20.753 | 50       | 0    | 0     | 0      | 7.003     |
| 50  | 20     | 99     | 50     | 18.574 | 23.611 | 50       | 0    | 0     | 0      | 34.102    |

**Table 8.** FourCell performances vs. LBCT4 using data3.

| $n$ | $pmin$ | $pmax$ | $\chi$ | MRPE   | WRPE   | $O_{21}$ | $O_{24}$ | CPU (sec) |
|-----|--------|--------|--------|--------|--------|----------|----------|-----------|
| 6   | 10     | 49     | 50     | 10.651 | 13.881 | 39       | 11       | 6.206     |
| 10  | 10     | 49     | 50     | 10.935 | 14.728 | 41       | 9        | 6.786     |
| 20  | 10     | 49     | 50     | 11.228 | 14.109 | 44       | 6        | 9.531     |
| 50  | 10     | 49     | 50     | 10.752 | 13.143 | 48       | 2        | 28.127    |

**Table 9.** FourCell performance vs. LBCT4 using data4.

| $n$ | $pmin$ | $pmax$ | $\chi$ | MRPE  | WRPE  | $O_{20}$ | $O'$ | $O''$ | $O'''$ | CPU (sec) |
|-----|--------|--------|--------|-------|-------|----------|------|-------|--------|-----------|
| 6   | 1      | 9      | 50     | 3.253 | 5.645 | 30       | 6    | 7     | 12     | 4.309     |
| 10  | 1      | 9      | 50     | 3.700 | 5.103 | 34       | 1    | 8     | 11     | 6.231     |
| 20  | 1      | 9      | 50     | 3.998 | 5.524 | 33       | 2    | 7     | 9      | 8.893     |
| 50  | 1      | 9      | 50     | 4.153 | 4.961 | 44       | 0    | 4     | 2      | 37.732    |

**Table 10.** FourCell performance vs. LBCT4 using data5.

| $n$ | $pmin$ | $pmax$ | $\chi$ | MRPE  | WRPE  | $O_1$ | $O'$ | $O''$ | $O'''$ | CPU (sec) |
|-----|--------|--------|--------|-------|-------|-------|------|-------|--------|-----------|
| 6   | 1      | 4      | 50     | 0.000 | 0.000 | 50    | 50   | 50    | 50     | 5.556     |
| 10  | 1      | 4      | 50     | 0.000 | 0.000 | 50    | 50   | 50    | 50     | 6.011     |
| 20  | 1      | 4      | 50     | 0.000 | 0.000 | 50    | 50   | 50    | 50     | 9.538     |
| 50  | 1      | 4      | 50     | 0.000 | 0.000 | 50    | 50   | 50    | 50     | 30.574    |

*Step 4:* Use Heuristic  $W_{cycle}$  to solve solve the part-sequencing problems for all  $W$ -cycles. Let  $C_j$  denote the cycle time for  $W$ -cycle  $S_j$ ,  $j \in \{2, 3, 5, 6, 7, 8, 12, 15, 16, 18, 19, 20, 21, 22, 23, 24\}$ .

*Step 5:* Use Heuristic GENIUS to solve the problems defined in Theorems 3, 4 and 5, and denote by  $C'$ ,  $C''$  and  $C'''$ , respectively, the corresponding cycle times.

*Step 6:* Find  $C_h = \min\{C_1, C_2, \dots, C_{24}, C', C'', C'''\}$  and its associated robot move cycle and part sequence. Terminate.

Due to the difficulty of finding optimal solutions to this problem, we tested the performance of **FourCell** against a lower bound found from Procedure **LBCT4** (see Appendix B), which is similar to Procedure **LBCT3**. The results of testing **FourCell** against **LBCT4** on 200 random problem instances generated using data1 (respectively, data3, data4, data5) appear in Table 7 (resp. Table 8, Table 9, Table 10). Much of the information in Tables 7 through 10 follows the format of our earlier results. For example, in column 7 (respectively, 8, 9, 10) of Table 7, we show  $O_{24}$  (resp.,  $O'$ ,  $O''$ ,  $O'''$ ), the number of instances for which the best cycle time found is given by  $C_{24}$  (resp.,  $C'$ ,  $C''$ ,  $C'''$ ). The mean

CPU time on a Pentium 60/90 MHz computer model over the  $\chi$  problem instances in each row is also shown, where  $\chi = 50$ . When robot activity times are small compared to processing times, as in data1 and data2, the machines are the bottlenecks. Hence, it is efficient for the robot to load the parts on each machine and move to perform other activities, as in cycle  $S_{24}$  and other  $W$ -cycles. Thus, cycle  $S_{24}$  dominates for data1 and data2. Since the results for data2 are similar to those for data1, we do not provide a table for data2. Note that  $S_{21}$  and  $S_{24}$  dominate for data3. However, when robot activity times are large compared to processing times, as in data4 and data5, one of the three combined cycles usually provides better solutions, because the robot can switch from one one-unit cycle to another. It is interesting to note in Table 9 that, for data4, cycle  $S_{20}$  also outperforms the combined cycles in some cases. In Table 10, the three combined cycles have identical performance for all the 200 problems tested, because processing times are very small compared to robot activity times in data5. Thus  $S_1$ , which is part of all the combined cycles, is most efficient. While all three combined cycles are listed as solutions in Table 10, in every case the solution used only cycle  $S_1$  for processing all the parts. Consequently, all the three combined cycles

had equal performance. Instances in data4 can be considered as intermediate cases between data3 and data5, since combined cycles and  $W$ -cycles compete for solutions. Heuristic solutions typically attain our lower bound when using data5, since the robot loads and waits at each machine for all parts, as in the first component of  $LB'_4$  in Procedure **LBCT4**, which tends to dominate the other lower bound components when processing times are small compared to robot activity times.

**REMARK 2.** The above development of Heuristic **FourCell** from Heuristic **ThreeCell** may be used as an induction step. The results of Sriskandarajah et al. (1998) provide a complete complexity mapping for cells of any size and an understanding of how robot move cycles proliferate when an extra machine is added to the cell. By using this understanding, as illustrated above, Heuristic **FourCell** can be extended to cells with any number of machines.

**REMARK 3.** In the above computational study, we assume that  $\delta_0 = 0.0$  and  $\gamma = 0.0$ . First,  $\delta_0 = 0.0$  means that there is no additional linear movement when the robot travels between  $M_i$  and  $M_{i+1}$ ,  $1 \leq i \leq m-1$ , after the rotational movement has been completed. This describes a robot-centered manufacturing cell where the robot is stationary at the center. Note that  $\gamma$  is the time saved by not stopping at an intermediate point during a robot movement. Moreover, it is always the case in practice that  $\gamma < \delta_i + \delta_{i+1}$  when the robot travels between  $M_{i-1}$  and  $M_{i+1}$ ,  $1 \leq i \leq m$ . Second,  $\gamma = 0.0$  implies that the robot travels at a constant speed and does not accelerate or decelerate when approaching or leaving a machine. The results in our computational study depend on the relative values of the robot activity times and the processing times. When the robot activity times are very small compared to processing times,  $\gamma$  and  $\delta_0$  are also very small. Alternatively, when the processing times are very small compared to robot activity times,  $\gamma$  and  $\delta_0$  are large. However, in both cases, the contributions from these parameters to the cycle times of various robot move cycles do not vary significantly between those cycles, and are thus not significant factors in determining which cycles are most efficient.

## 5. A CELL DESIGN PROBLEM

The cell design problem considered here involves organizing several machines into  $R \geq 2$  cells, which are arranged in a serial production process with intermediate buffers. Black and Schroer (1993) state: "The next generation of American factories (the factories with a future) will be designed with manufacturing and assembly cells linked together with a pull system for material and information control." We consider the combined problem of dividing machines into cells, determining robot move cycles and sequencing parts. Where large buffers permit complete resequencing between cells, we propose Heuristic **CellDesign1** in which a partition, a robot move

cycle, and a part sequence in each cell are found by heuristically minimizing the cycle time of each cell individually. In **CellDesign1**, two-machine cells are solved using Improved MinCycle, and three-machine cells are solved using Heuristic **ThreeCell**.

Where no resequencing of parts between the cells is possible, we propose Heuristic **CellDesign2**. For each possible partition, the part sequencing problem is solved in each of the  $R$  cells, to find (locally) optimal sequences  $\sigma_1, \dots, \sigma_R$  and a corresponding robot move cycle for each cell. If  $\sigma_1 = \dots = \sigma_R$ , then that sequence is optimal for the overall problem. Otherwise, **CellDesign2** solves the overall problem  $R$  times by imposing on all the cells the sequence  $\sigma_i$  and finding a best robot move cycle for each cell, for  $i = 1, \dots, R$ . For a given partition, the sequence found by **CellDesign2** is a heuristic solution to the problem where resequencing is not possible. The partition and common part sequence that jointly give the lowest overall cycle time are chosen.

Procedures **CellDesign1** and **CellDesign2**, which are described in more detail by Kamoun et al. (1997), were coded in the Turbo Pascal language and executed on a Pentium 60/90 MHz computer model. We tested the performance of these procedures against a lower bound, **LBCTD**, on the optimal cycle time. For a given partition, this lower bound evaluates two-machine cells using Improved MinCycle, and three-machine cells using Procedure **LBCT3**. As in Miller and Walker's (1990) example from Massey Ferguson, our test problem has eight machines, which are being divided into three cells. The results of testing **CellDesign1** (respectively, **CellDesign2**) on 10 randomly generated problem instances each from data1, data2, data4, and data5 appear in Table 11 (resp., Table 12). Data set data3 is not relevant here, since it is designed to test the effect of smaller variance in processing times on the last machine in a cell. Much of the information in Tables 11 and 12 follows the format of Tables 1 and 2. In addition, in column 6 (respectively, 7) we provide MRPI (resp., HRPI), the mean (resp., highest) relative percentage improvement compared to the cycle time of the worst partition and sequence considered in the procedure, computed over the 10 instances studied. The results show that our heuristics find good approximate solutions, especially for instances from data4 and data5.

## 6. OPEN PROBLEMS

Here we provide a list of open research problems that arise in scheduling robotic cells.

(R1):  $MRCm|k = 1|C_i$ : until recently, it remained open for  $m \geq 4$  whether it is optimal to repeat a single one-unit cycle, or whether the average cycle time of an MPS can be reduced using  $q$ -unit cycles with  $q > 1$ . For  $m = 2$  or  $m = 3$ , the optimal solution is given by one-unit cycles, see Sethi et al. (1992), van de Klundert (1996), and Brauner and Finke (1999a). For  $m \geq 4$ , Brauner and Finke (1999b) show that a  $q$ -unit cycle with  $q > 1$  can provide a cycle time that is smaller than the smallest 1-unit cycle time.

**Table 11. CellDesign1 performance vs. LBCD.**

| $n$ | Dataset | $\chi$ | MRPE   | WRPE   | MRPI  | HRPI  | CPU (sec) |
|-----|---------|--------|--------|--------|-------|-------|-----------|
| 50  | data1   | 10     | 13.551 | 16.489 | 1.583 | 3.996 | 30.424    |
| 50  | data2   | 10     | 15.381 | 17.115 | 1.314 | 3.056 | 31.919    |
| 50  | data4   | 10     | 3.409  | 4.383  | 0.217 | 1.038 | 31.207    |
| 50  | data5   | 10     | 0.000  | 0.000  | 0.581 | 1.266 | 33.980    |

**Table 12. CellDesign2 performance vs. LBCD.**

| $n$ | Dataset | $\chi$ | MRPE   | WRPE   | MRPI  | HRPI  | CPU (sec) |
|-----|---------|--------|--------|--------|-------|-------|-----------|
| 50  | data1   | 10     | 16.470 | 18.763 | 5.695 | 9.111 | 32.007    |
| 50  | data2   | 10     | 16.045 | 16.961 | 2.925 | 3.769 | 35.521    |
| 50  | data4   | 10     | 3.660  | 4.541  | 1.645 | 2.672 | 35.342    |
| 50  | data5   | 10     | 0.000  | 0.000  | 0.581 | 1.266 | 39.437    |

(R2):  $MRCm|k \geq 2|C_i$  with fixed  $k$ : the complexity of this problem remains open.

(R3): In  $MRC3|k \geq 2, 1 - unit|C_i$ , robot move cycles  $S_{2,3}$  and  $S_{6,3}$  define intractable part sequencing problems (see Hall et al. 1998). It would be interesting to study the worst-case performance of Heuristics  $S_2$ cycle and  $S_6$ cycle described in §2.

(R4): It is important to investigate how allowing buffer storage within a cell may improve cycle time. The study of systems without buffers often provides useful intuition for systems with limited buffers, as discussed by Hall and Sriskandarajah (1996).

(R5): In this paper, we consider a robot with a single gripper. An alternative specification is that the robot, equipped with a dual gripper, can at a single visit to a machine, both unload a finished part and load a new part. Another possible assumption is that robot load and unload times depend upon the identity of the part being handled.

(R6): The joint robot move and part sequencing problem in a similar openshop or jobshop environment provides several open research issues.

(R7): The computational complexity of scheduling problems in which several parallel machines share a common server is another topic worthy of investigation. Hall et al. (1999) provide algorithmic and intractability results, but several problems remain open.

## 7. CONCLUSIONS

In this paper we describe and test practically useful methods for the scheduling of robotic cells to minimize cycle time. We also show how our scheduling methods can be used to address design issues that arise where several machines need to be arranged into cells with intermediate buffers. Most of the important theoretical and practical issues arising in the design and scheduling of robotic cells have now been completely or partially resolved. We indicate in §6 some open problems for future research.

Manufacturing systems often use a variety of material handling devices for loading, unloading, setup, and

movement of parts between machines. It is important to understand the relationships between scheduling problems arising in different material handling environments, since this understanding may provide insights and useful results. For example, if a bidirectional AGV is used to serve machines that are arranged in a line layout, the results of our study are directly applicable to the scheduling of AGV-served manufacturing systems. Ganesharajah et al. (1998) review the literature of design and operational issues in AGV systems. Ganesharajah et al. (1999a,b) describe a variety of solution procedures for scheduling, fleet sizing and decomposition issues in AGV systems.

Some comments about possible alternative objectives now follow. In a traditional flowshop environment, a frequently used objective is the minimization of the time to complete a batch of jobs, or makespan. This objective becomes relevant to the robotic cell environment when the parts to be processed arrive in nonidentical batches. For example, Kise et al. (1991) consider such problems in an AGV-served flowshop. Other objectives involving due dates for completed jobs could also be studied. We hope that our work will stimulate researchers to consider different objectives as well as cycle time minimization for scheduling in automated material handling environments.

## APPENDIX A

### CONSTRUCTION OF ROBOT MOVE CYCLES IN FOUR MACHINE CELLS

$$S_{1,4} : \{M_4^+, M_1^-, M_2^-, M_3^-, M_4^-, M_4^+\},$$

$$S_{2,4} : \{M_4^+, M_1^-, M_2^-, M_4^-, M_3^-, M_4^+\},$$

$$S_{3,4} : \{M_4^+, M_1^-, M_4^-, M_2^-, M_3^-, M_4^+\},$$

$$S_{4,4} : \{M_4^+, M_4^-, M_1^-, M_2^-, M_3^-, M_4^+\},$$

$$S_{5,4} : \{M_4^+, M_1^-, M_3^-, M_2^-, M_4^-, M_4^+\},$$

$$S_{6,4} : \{M_4^+, M_1^-, M_3^-, M_4^-, M_2^-, M_4^+\},$$

$$S_{7,4} : \{M_4^+, M_1^-, M_4^-, M_3^-, M_2^-, M_4^+\},$$

$$S_{8,4} : \{M_4^+, M_4^-, M_1^-, M_3^-, M_2^-, M_4^+\},$$

- $S_{9,4} : \{M_4^+, M_3^-, M_1^-, M_2^-, M_4^-, M_4^+\},$   
 $S_{10,4} : \{M_4^+, M_3^-, M_1^-, M_4^-, M_2^-, M_4^+\},$   
 $S_{11,4} : \{M_4^+, M_3^-, M_4^-, M_1^-, M_2^-, M_4^+\},$   
 $S_{12,4} : \{M_4^+, M_4^-, M_3^-, M_1^-, M_2^-, M_4^+\},$   
 $S_{13,4} : \{M_4^+, M_2^-, M_3^-, M_1^-, M_4^-, M_4^+\},$   
 $S_{14,4} : \{M_4^+, M_2^-, M_3^-, M_4^-, M_1^-, M_4^+\},$   
 $S_{15,4} : \{M_4^+, M_2^-, M_4^-, M_3^-, M_1^-, M_4^+\},$   
 $S_{16,4} : \{M_4^+, M_4^-, M_2^-, M_3^-, M_1^-, M_4^+\},$   
 $S_{17,4} : \{M_4^+, M_2^-, M_1^-, M_3^-, M_4^-, M_4^+\},$   
 $S_{18,4} : \{M_4^+, M_2^-, M_1^-, M_4^-, M_3^-, M_4^+\},$   
 $S_{19,4} : \{M_4^+, M_2^-, M_4^-, M_1^-, M_3^-, M_4^+\},$   
 $S_{20,4} : \{M_4^+, M_4^-, M_2^-, M_1^-, M_3^-, M_4^+\},$   
 $S_{21,4} : \{M_4^+, M_3^-, M_2^-, M_1^-, M_4^-, M_4^+\},$   
 $S_{22,4} : \{M_4^+, M_3^-, M_2^-, M_4^-, M_1^-, M_4^+\},$   
 $S_{23,4} : \{M_4^+, M_3^-, M_4^-, M_2^-, M_1^-, M_4^+\},$   
 $S_{24,4} : \{M_4^+, M_4^-, M_3^-, M_2^-, M_1^-, M_4^+\}.$

## APPENDIX B

### A PROCEDURE TO OBTAIN A LOWER BOUND IN FOUR MACHINE CELLS

**Procedure LBCT4** ( $a_i, b_i, c_i, d_i, i = 1, \dots, n, \delta_0, \delta_1, \delta_2, \delta_3, \delta_4, \delta_5, \gamma$ )

*Step 1:* Call MinCycle ( $a_i, b_i, i = 1, \dots, n, \delta_0, \delta_1, \delta_2, \delta_3, \gamma$ ), and let  $C^*$  denote the resulting cycle time.

$$LB'_1 = C^*.$$

Call MinCycle ( $b_i, c_i, i = 1, \dots, n, \delta_0, \delta_2, \delta_3, \delta_4, \gamma$ ), and let  $C^*$  denote the resulting cycle time.

$$LB'_2 = C^*.$$

Call MinCycle ( $c_i, d_i, i = 1, \dots, n, \delta_0, \delta_3, \delta_4, \delta_5, \gamma$ ), and let  $C^*$  denote the resulting cycle time.

$$LB'_3 = C^*.$$

$$LB'_4 = n\alpha'_1 + \sum_{i=1}^n \min\{a_i + b_i + c_i + d_i, \min\{a_i + b_i + c_i + \delta_0 + \delta_4, a_i + b_i + d_i + \delta_0 + \delta_{3,4}, a_i + c_i + d_i + \delta_0 + \delta_{2,3}, b_i + c_i + d_i + \delta_0 + \delta_2\}, \min\{a_i + b_i + 2\delta_0 + \delta_{3,4} + \delta_4, a_i + c_i + 2\delta_0 + \delta_{2,3} + \delta_4, a_i + d_i + 2\delta_0 + \delta_{2,3} + \delta_{3,4}, b_i + c_i + 2\delta_0 + \delta_2 + \delta_4, b_i + d_i + 2\delta_0 + \delta_2 + \delta_{3,4}, c_i + d_i + 2\delta_0 + \delta_2 + \delta_{2,3}\}, \min\{a_i + 3\delta_0 + \delta_{2,3} + \delta_{3,4} + \delta_4, b_i + 3\delta_0 + \delta_2 + \delta_{3,4} + \delta_4, c_i + 3\delta_0 + \delta_2 + \delta_{2,3} + \delta_4, d_i + 3\delta_0 + \delta_2 + \delta_{2,3} + \delta_{3,4}\}, 4\delta_0 + \delta_2 + \delta_{2,3} + \delta_{3,4} + \delta_4\}, \text{ where } \delta_{i,j} = \min\{\delta_i, \delta_j\} \text{ and } \alpha'_1 = 6\delta_0 + 2\sum_{i=1}^5 \delta_i + \sum_{i=1}^{10} \varepsilon_i - 4\gamma.$$

*Step 2:*  $LB = \max\{LB'_1, LB'_2, LB'_3, LB'_4\}$ . Terminate.

## ACKNOWLEDGMENTS

This research was supported by Natural Sciences and Engineering Research Council of Canada (Grant no. OGP

0104900) and by the Summer Fellowship Program, Fisher College of Business, The Ohio State University. The authors thank Professor Gilbert Laporte and CRT (University of Montreal) for the use of their GENIUS program for the traveling salesman problem. An associate editor and a referee provided helpful comments that improved the presentation of the results.

## REFERENCES

- Aneja Y.P., H. Kamoun. 1999. Scheduling of parts and robot activities in two-machine robotic cell. *Comput. Oper. Res.*, **26** 297–312.
- Asfahl, C.R. 1985. *Robots and Manufacturing Automation*. John Wiley and Sons, New York.
- Askin, R.G., C.R. Standridge. 1993. *Modeling and Analysis of Manufacturing Systems*. John Wiley and Sons, New York.
- Black, J.T. 1983. Cellular manufacturing systems reduce setup time, make small lot production economical. *Indust. Eng. (November)* 36–48.
- , B.J. Schroer. 1993. Simulation of an apparel assembly cell with walking workers and decouplers. *J. Mfg. Sys.* **12** 170–180.
- Brauner, N., G. Finke. 1999a. On the conjecture in robotic cells: new simplified proof for the three-machine case. *INFOR* **37** 20–36.
- Brauner, N., G. Finke. 1999b. On cycles and permutations in robotic cells, Technical Report, RR 1010-I, Laboratoire LEIBNIZ, Institut IMAG, Grenoble, France.
- Campbell, H.G., R.A. Dudek, M.L. Smith. 1970. A heuristic algorithm for the  $n$  job  $m$  machine sequencing problem. *Management Sci.* **16** B630–B637.
- Crama, Y. 1997. Combinatorial optimization models for production scheduling in automated manufacturing systems. *Euro. J. Oper. Res.* **99** 136–153.
- , J. van de Klundert. 1997. Cyclic scheduling of identical parts in a robotic cell. *Oper. Res.* **45** 952–965.
- Drezner, Z., S.Y. Nof. 1984. Optimal bin picking and insertion plans for assembly robots. *IIE Trans.* **16** 262–270.
- Ganesharajah, T., N.G. Hall, C. Sriskandarajah. 1998. Design and operational issues in AGV-served manufacturing systems. *Ann. Oper. Res.* **76** 109–154.
- , —, —. 1999a. Operational decisions in AGV-served flowshop loops: scheduling. Working paper, Fisher College of Business, The Ohio State University, revised for publication.
- , —, —. 1999b. Operational decisions in AGV-served flowshop loops: fleet sizing and decomposition. Working paper, Fisher College of Business, The Ohio State University, revised for publication.
- Garey, M.R., D.S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman and Company, San Francisco.
- Gendreau, M., A. Hertz, G. Laporte. 1992. New insertion and postoptimization procedures for the traveling salesman problem. *Oper. Res.* **40** 1086–1094.
- Gilmore, P.C., R.E. Gomory. 1964. Sequencing a one state-variable machine: a solvable case of the travelling salesman problem. *Oper. Res.* **12** 655–679.

- Graham, R.L., E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan. 1979. Optimization and approximation in deterministic sequencing and scheduling: a survey. *Ann. Discrete Math.* **5** 287–326.
- Hall, N.G. 1999. Operations research techniques for robotics systems. *Handbook of Industrial Robotics*, vol. II, S.Y. Nof, ed. John Wiley, New York, 543–577.
- , H. Kamoun, C. Sriskandarajah. 1997. Scheduling in robotic cells: classification, two and three machine cells. *Oper. Res.* **45** 421–439.
- , ———. 1998. Scheduling in robotic cells: complexity and steady state analysis. *Euro. J. Oper. Res.* **109** 43–65.
- , C.N. Potts, C. Sriskandarajah. 1999. Parallel machine scheduling with a common server. *Disc. App. Math.*, to appear.
- , C. Sriskandarajah. 1996. A survey of machine scheduling problems with blocking and no-wait in process. *Oper. Res.* **44** 510–525.
- Hartley, J. 1983. *Robots at Work*, North-Holland Publishing Company, New York.
- Kamoun, H., N.G. Hall, C. Sriskandarajah. 1997. Scheduling in robotic cells: heuristics and cell design. Working Paper (revised September 1997), Department of Mechanical and Industrial Engineering, University of Toronto.
- Kise, H., T. Shioyama, T. Ibaraki. 1991. Automated two-machine flowshop scheduling: a solvable case. *IIE Trans.* **23** 10–16.
- Logendran, R., C. Sriskandarajah. 1996. Sequencing of robot activities and parts in two machine robotic cells. *Internat. J. Production Res.* **34** 3447–3463.
- Matsuo, H. 1990. Cyclic sequencing problems in a two machine permutation flowshop: complexity, worst case and average case analysis. *Naval Res. Logist.* **37** 679–694.
- Miller, R.K., T.C. Walker. 1990. *FMS/CIM Systems Integration Handbook*. The Fairmont Press, Lilburn, GA.
- Nof, S.Y., Z. Drezner. 1986. Part flow in the robotic assembly plan problem. *Robotics and material flow* S.Y. Nof, (Ed.) Elsevier Press, New York.
- Sethi, S.P., C. Sriskandarajah, G. Sorger, J. Blazewicz, W. Kubiak. 1992. Sequencing of parts and robot moves in a robotic cell. *Internat. J. Flexible Mfg. Syst.* **4** 331–358.
- Sriskandarajah, C., N.G. Hall, H. Kamoun. 1998. Scheduling large robotic cells without buffers. *Ann. Oper. Res.* **76** 287–321.
- Tompkins, J.A., J.A. White, Y.A. Bozer, E.H. Frazelle, J.M.A. Tanchoco, J. Trevino. 1996. *Facilities Planning*. 2nd ed., John Wiley and Sons, New York.
- van de Klundert, J. 1996. Scheduling problems in automated manufacturing. Ph. D. Dissertation no. 96-35. Faculty of economics and business administration, University of Limburg, Maastricht, The Netherlands.
- Wilhelm, W.E. 1987. Complexity of sequencing tasks in assembly cells attended by one or two robots. *Naval Res. Logist.* **34** 721–738.
- , S.C. Sarin. 1985. A structure for sequencing robot activities in machine loading applications. *Internat. J. Production Res.* **23** 47–64.
- Wittrock, R.J. 1985. Scheduling algorithms for flexible flow lines. *IBM J. Res. Development* **29** 401–412.